

# Machine Learning for Systems and Control

5SC28

Lecture 6

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group

Department of Electrical Engineering

Eindhoven University of Technology

Academic Year: 2025-2026 (version 12/05/2026)

# Learning Outcomes

Data-Driven Modelling → previous lectures

- **Explain, discuss, compare and interpret** the main techniques and theory for reinforcement learning based control starting from **classical Q-learning** up to **actor-critic** and **model internalization**-based methods;
- **Recommend and evaluate** a machine learning method for a real-life application in a systems and **control setting**.
- **Implement, tailor, and apply** the Gaussian process, (deep) neural network and **reinforcement learning techniques** for model and control learning on real-world systems (e.g. on an inverted pendulum laboratory setup).

# Reinforcement Learning

What is Reinforcement Learning?

Multi-Armed Bandits

Finite Markov Decision Process

# Reinforcement Learning

What is Reinforcement Learning?

Multi-Armed Bandits

Finite Markov Decision Process

# What is Reinforcement Learning?

Learn by interacting with the environment

# What is Reinforcement Learning?

Learn by interacting with the environment

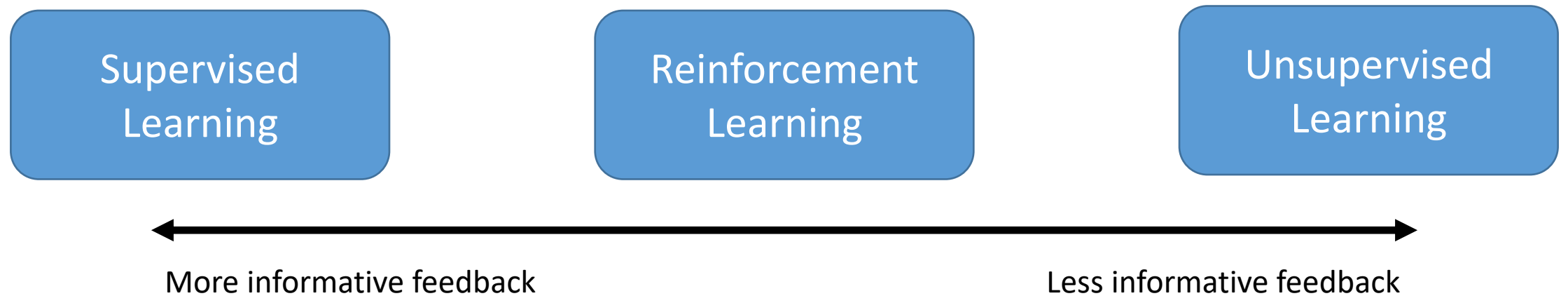
Closed loop – awareness of the response to your action

Exploring: cause – effect

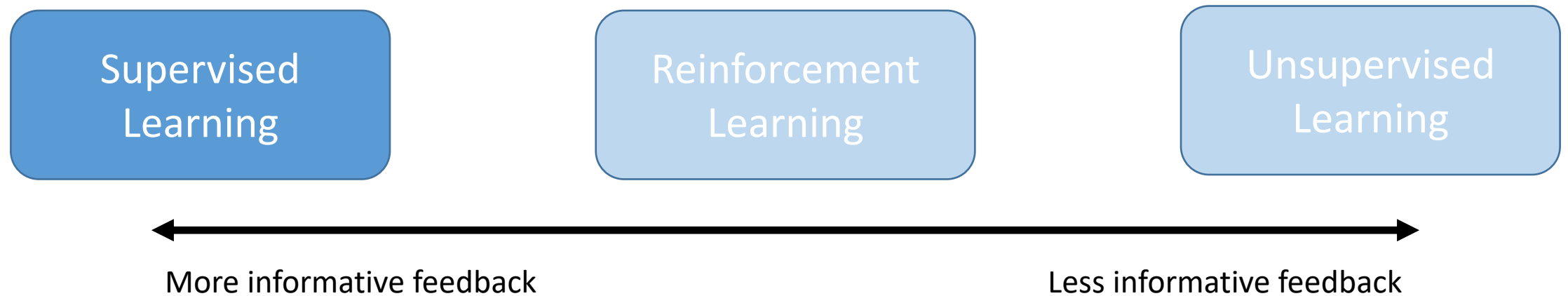
Goal directed learning

No explicit teacher

# What is Reinforcement Learning?



# What is Reinforcement Learning?

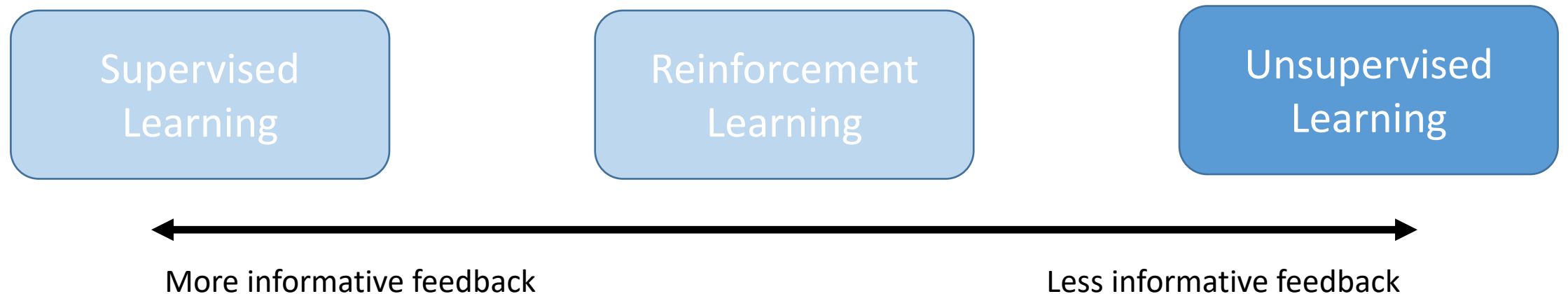


Inputs and outputs are known  
Infer input-output relationship

e.g. function estimation



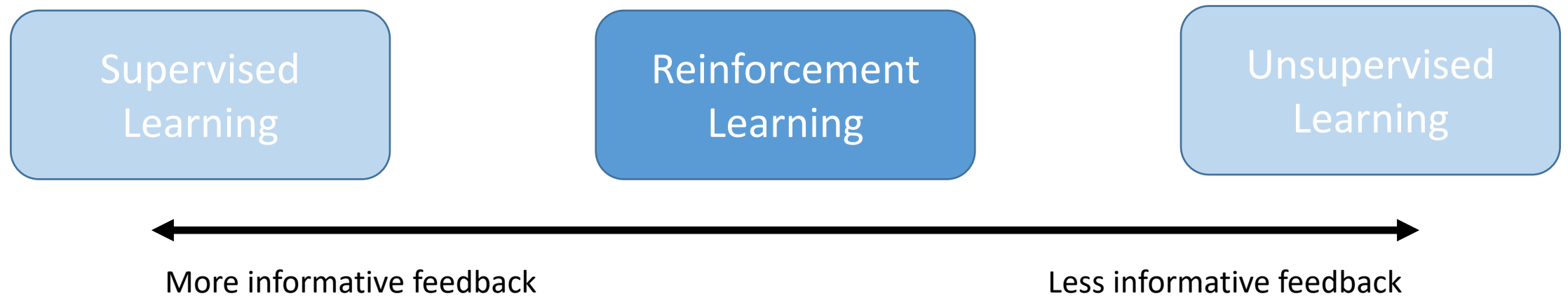
# What is Reinforcement Learning?



Only the inputs are known  
Find patterns and features from data

e.g. clustering

# What is Reinforcement Learning?



Correct outputs not available, only rewards  
Find optimal policy / behavior / controller

# What is Reinforcement Learning?

*“Reinforcement learning is a **computational approach** to understanding and **automating goal-directed learning** and decision-making. It is distinguished from other computational approaches by its **emphasis on learning by an agent from direct interaction with its environment**, without relying on exemplary supervision or complete models of the environment.”<sup>1</sup>*

<sup>1</sup> R.S. Sutton and A.G. Barto, “Reinforcement Learning: An Introduction”, The MIT Press, 2018

# What is Reinforcement Learning?

*“Reinforcement learning is a **computational approach** to understanding and **automating goal-directed learning** and decision-making. It is distinguished from other computational approaches by its **emphasis on learning by an agent from direct interaction with its environment**, without relying on exemplary supervision or complete models of the environment.”<sup>1</sup>*

Reinforcement learning is about adaptive, model-free control

<sup>1</sup> R.S. Sutton and A.G. Barto, “Reinforcement Learning: An Introduction”, The MIT Press, 2015

# Reinforcement Learning

What is Reinforcement Learning?

**Multi-Armed Bandits**

Finite Markov Decision Process

# One-Armed Bandit



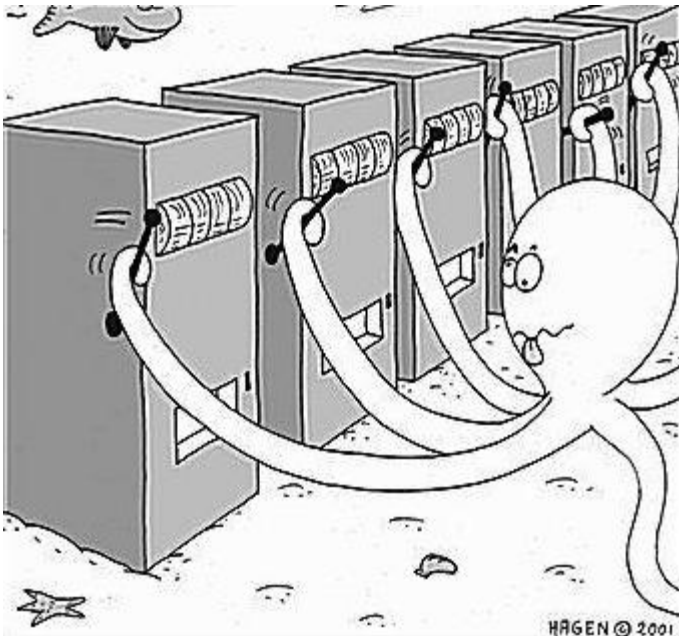
1. Pull the lever

2. Get a reward  $r$  with some probability  $f_i(r)$

Source: <https://www.partycity.com/>

# Multi-Armed Bandits

Multiple slot machines, each with their own unknown probability density function  $f_i(r)$   
You will play for a certain time (N lever pulls).

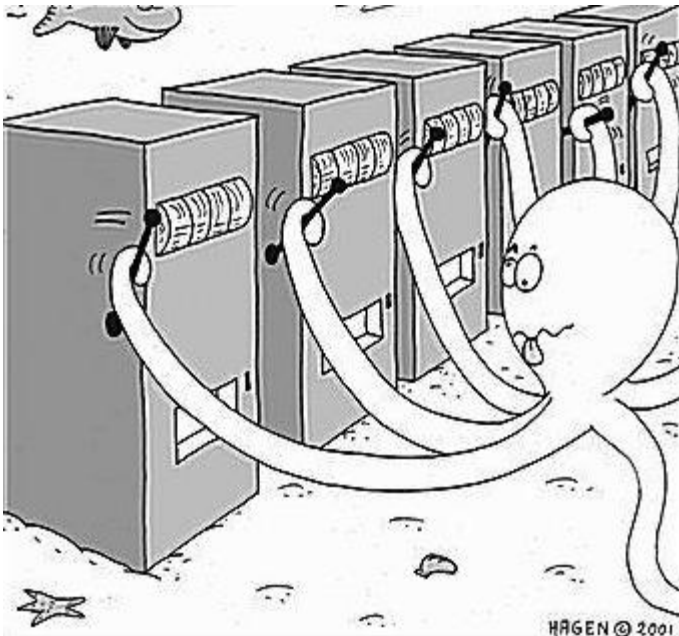


Source: <https://www.analyticsvidhya.com/>

# Multi-Armed Bandits

Multiple slot machines, each with their own unknown probability density function  $f_i(r)$   
You will play for a certain time (N lever pulls).

How to maximize the reward?  
Which levers to pull?



Source: <https://www.analyticsvidhya.com/>



# Multi-Armed Bandits

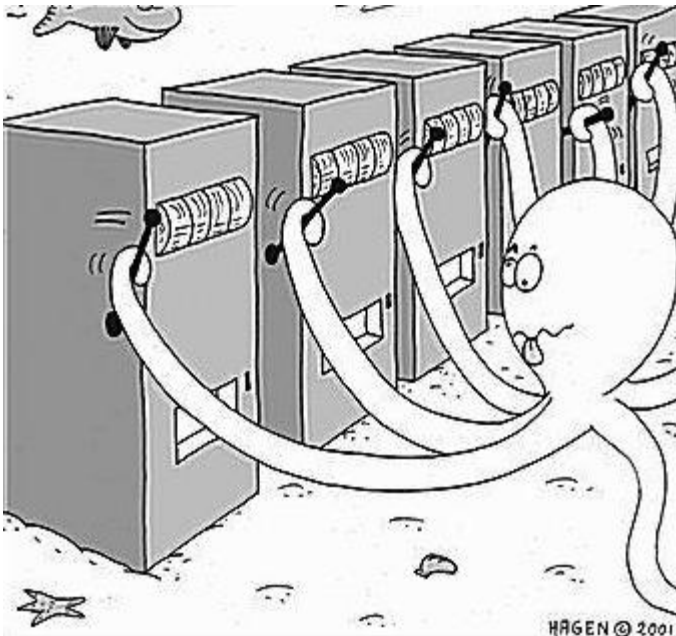
Multiple slot machines, each with their own unknown probability density function  $f_i(r)$   
You will play for a certain time (N lever pulls).

How to maximize the reward?  
Which levers to pull?

## Exploration vs Exploitation

Explore the reward given by the different slot machines?

Exploit the maximal reward currently known?



Source: <https://www.analyticsvidhya.com/>

# Practical Relevance

## Online Advertising

- Explore interests of user
- Exploit clicks of the user

## Network Routing

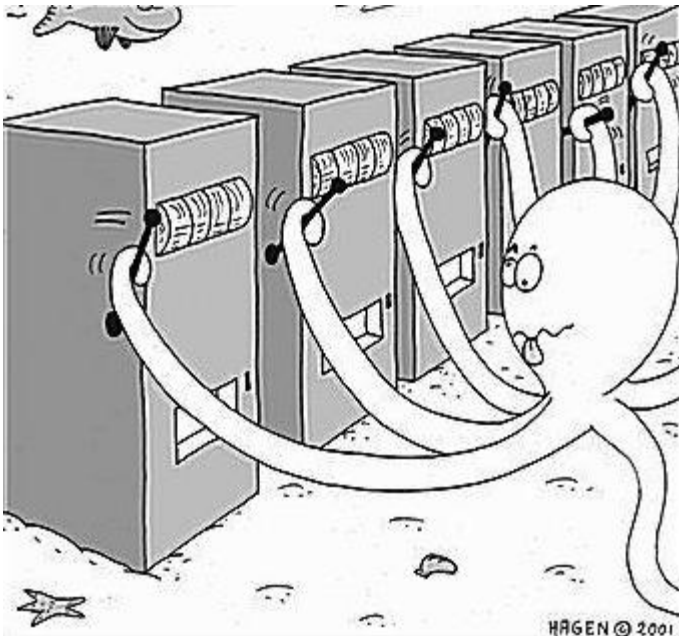
- Allocate the best channel (maximize throughput)
- Monitor channel quality

## Clinical Trials

- Try different treatments
- Give the best treatment

## Going out for dinner

- Try a new place
- Go back to your favorite



Source: <https://www.analyticsvidhya.com/>

# Reinforcement Learning Relevance

Agent interacts with the environment through **Actions**

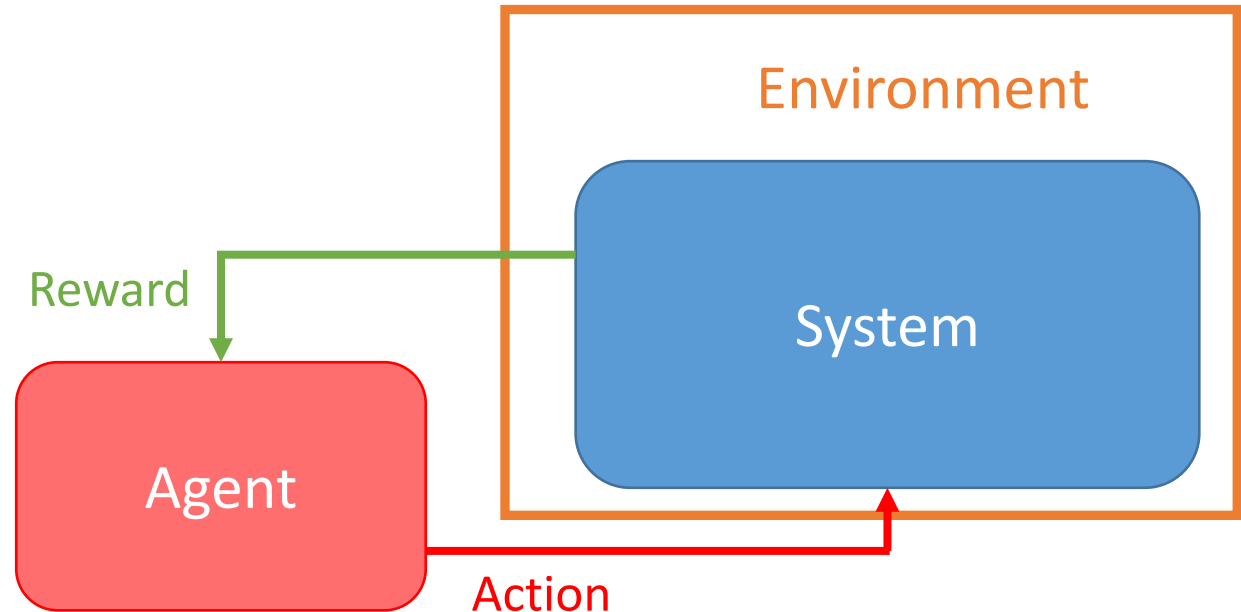
Receive **Reward** as a performance feedback

**Action:** Pull a lever

**Reward:** The return of the bandit

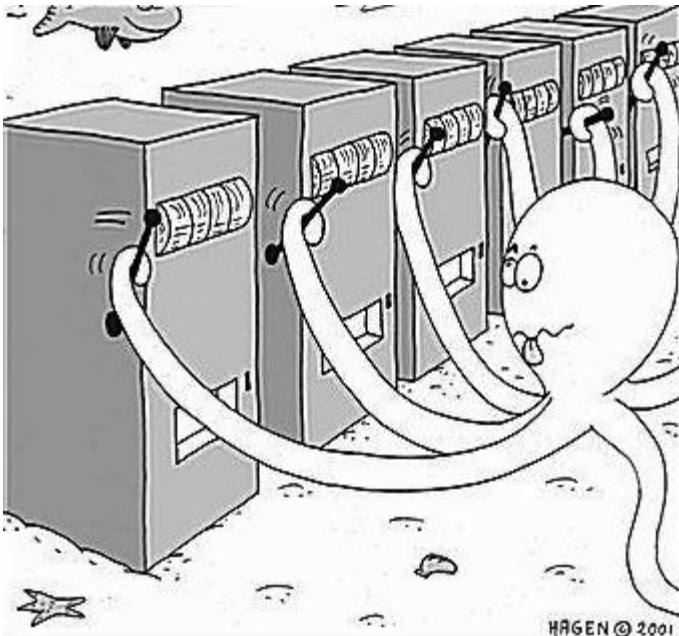
**Goal:** Maximize cumulative reward

Illustrates the exploration – exploitation trade-off



# Problem Setting

$K$  slot machines, each with their own unknown probability density function  $f_i(r)$   
You will play for a certain time ( $N$  lever pulls).



Source: <https://www.analyticsvidhya.com/>

Rewards  $r$ : 0 or 1

$P_i$  : probability of machine  $i$  resulting in a reward  $r = 1$

The value of action  $u \in U$  is the expected reward:

$$Q(u) = \mathbb{E}\{r|u\}$$

Goal: maximize cumulative reward  $\sum_{k=1}^N r_k$

# Maximal Reward

Always pull the lever with the highest expected value

Optimal reward probability:

$$P^* = Q(u^*) = \max_{u \in U} Q(u) = \max_{1 \leq i \leq K} P_i$$

# Goal

Always pull the lever with the highest expected value

Optimal reward probability:

$$P^* = Q(u^*) = \max_{u \in U} Q(u) = \max_{1 \leq i \leq K} P_i$$

Reformulated Goal: minimize cost function

$$V_N = \mathbb{E} \left\{ \sum_{k=1}^N \underbrace{(P^* - Q(u_k))}_{\text{regret}} \right\}$$

Problem:  $P_i$  is unknown!

# Exploitation - Exploration

Goal: minimize cost function

$$V_N = \mathbb{E} \left\{ \sum_{k=1}^N \underbrace{(P^* - Q(a_k))}_{\text{regret}} \right\}$$

Problem:  $P_i$  is unknown!

Dilemma:

1. Minimize immediate regret at time  $t$  (greedy action / exploitation)
2. Improve estimates of  $P_i$  (exploration)

# Action Value Function

Define the action value function at time  $k$  as:

$$Q_k(u) = \frac{\sum_{i=1}^k r_i 1_{u_i=u}}{\sum_{i=1}^k 1_{u_i=u}} \quad \rightarrow \quad \frac{\text{Total reward obtained with action } u}{\text{Total times action } u \text{ is chosen}}$$

This function is an estimate of  $Q(u)$  using the observations available at time  $k$ .



# Strategy 1: Greedy Approach

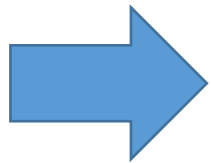
$Q_0(u)$  is set at a user-chosen initial value.

Always choose the action for which  $Q_k(u)$  is maximal:  $u_{k+1} = \arg \max_u Q_k(u)$

# Strategy 1: Greedy Approach

$Q_0(u)$  is set at a user-chosen initial value.

Always choose the action for which  $Q_k(u)$  is maximal:  $u_{k+1} = \arg \max_u Q_k(u)$



**No exploration!**

# Strategy 2: $\epsilon$ - Greedy Approach

$Q_0(u)$  is set at a user-chosen initial value.

Choose the action for which  $Q_k(u)$  is maximal with probability  $(1 - \epsilon)$

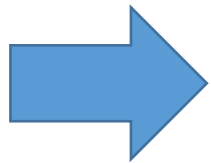
Sample randomly one of the other actions with probability  $\epsilon$

# Strategy 2: $\epsilon$ - Greedy Approach

$Q_0(u)$  is set at a user-chosen initial value.

Choose the action for which  $Q_k(u)$  is maximal with probability  $(1 - \epsilon)$

Sample randomly one of the other actions with probability  $\epsilon$



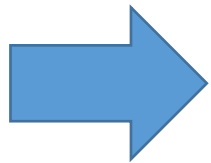
**Exploration!**

# Strategy 2: $\epsilon$ - Greedy Approach

$Q_0(u)$  is set at a user-chosen initial value.

Choose the action for which  $Q_k(u)$  is maximal with probability  $(1 - \epsilon)$

Sample randomly one of the other actions with probability  $\epsilon$



## **Exploration!**

But we keep exploring even long after the estimates  $Q_k(u)$  converged to their expected values  $Q(u)$

# Recursive Implementation

If action  $u$  is taken:

$$Q_k(u) = \frac{\sum_{i=1}^k r_i 1_{u_i=u}}{\sum_{i=1}^k 1_{u_i=u}} = Q_{k-1}(u) + \frac{1}{N_u} (r_k - Q_{k-1}(u))$$

Action result  
↑  
↓  
Number of times  
action  $u$  is taken

# Recursive Implementation

If action  $u$  is taken:

$$Q_k(u) = \frac{\sum_{i=1}^k r_i 1_{u_i=u}}{\sum_{i=1}^k 1_{u_i=u}} = Q_{k-1}(u) + \frac{1}{N_u} (r_k - Q_{k-1}(u))$$

$$X_k = \frac{1}{k} \sum_{i=1}^k x_i$$

=

Exercise 1a

=

$$= X_{k-1} + \frac{1}{k} (x_k - X_{k-1})$$

# Strategy 2: $\epsilon$ - Greedy Algorithm

**Parameters:** exploration probability  $\epsilon \in [0, 1]$

**Initialize**  $Q_0(u) = 0$ ,  $N_u = 0$

**Repeat** forever

$$u \leftarrow \begin{cases} \arg \max_u Q(u) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$$

Exploitation  
Exploration

Take action  $u$  and observe the reward  $r$

$$N_u \leftarrow N_u + 1$$

$$Q(u) \leftarrow Q(u) + \frac{1}{N_u} (r - Q(u))$$



# Example

$$P_1 = 0.5$$



$$P_2 = 0.4$$



$$P_3 = 0.7$$

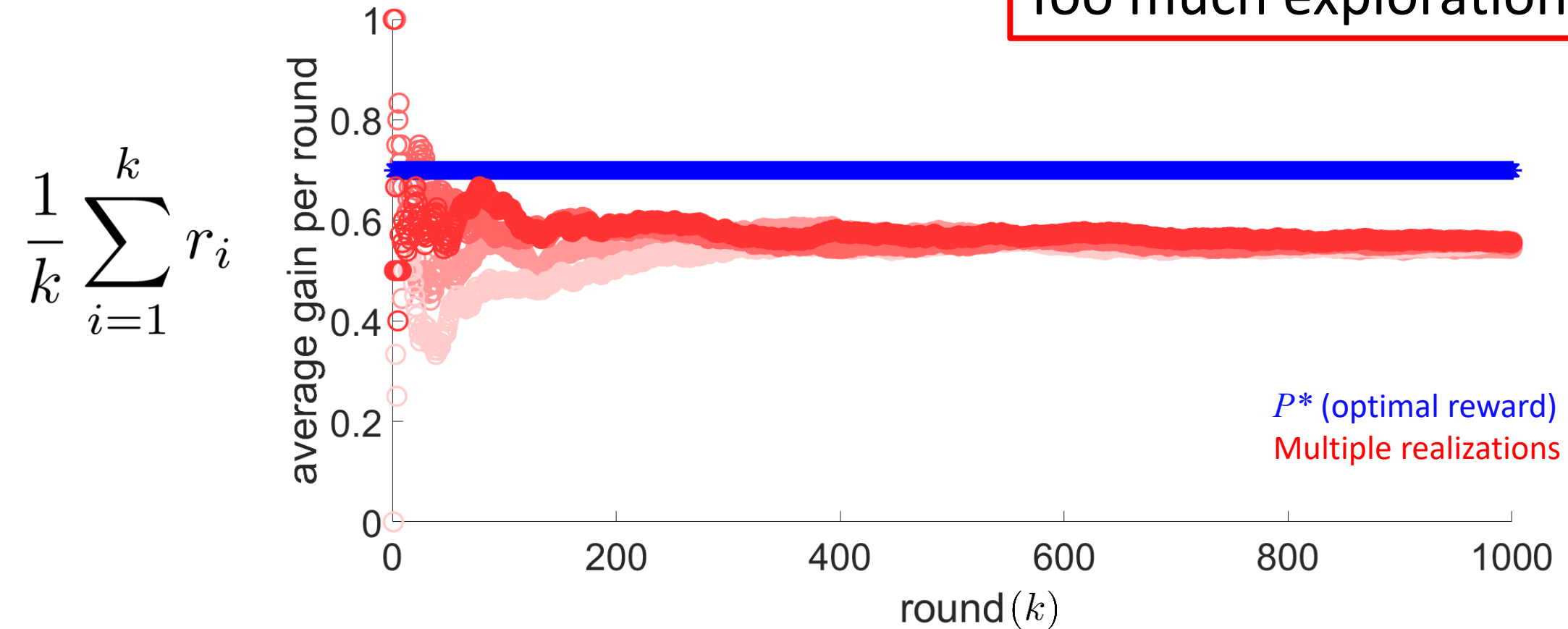


$$P_4 = 0.6$$



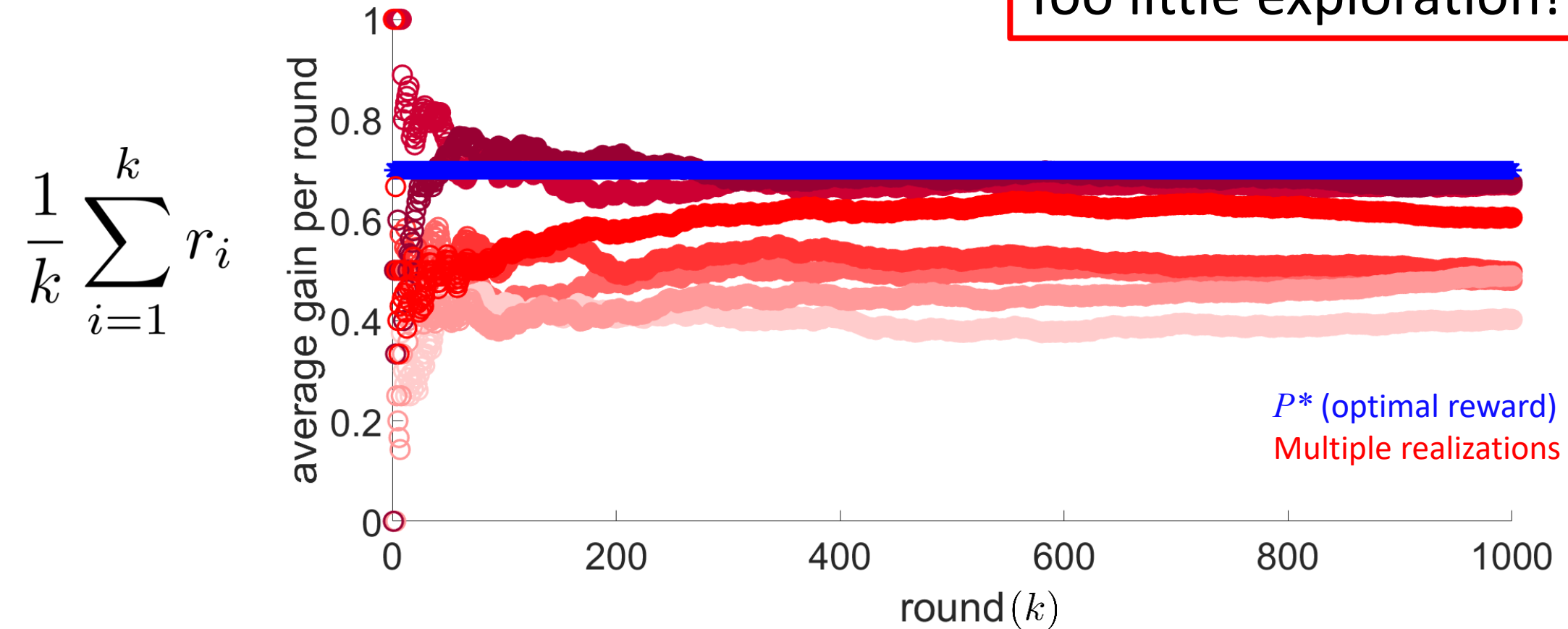
# Example: Random Choices

Too much exploration!



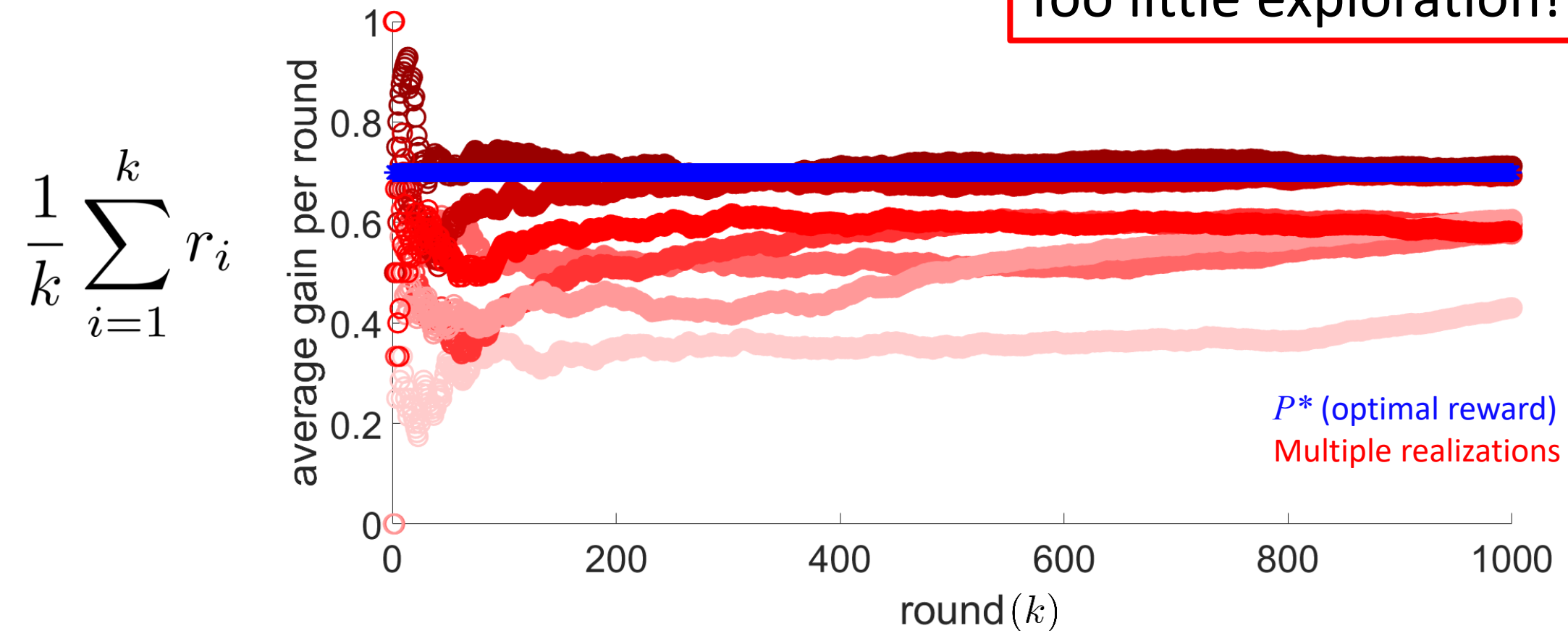
# Example: Greedy Choice

Too little exploration!



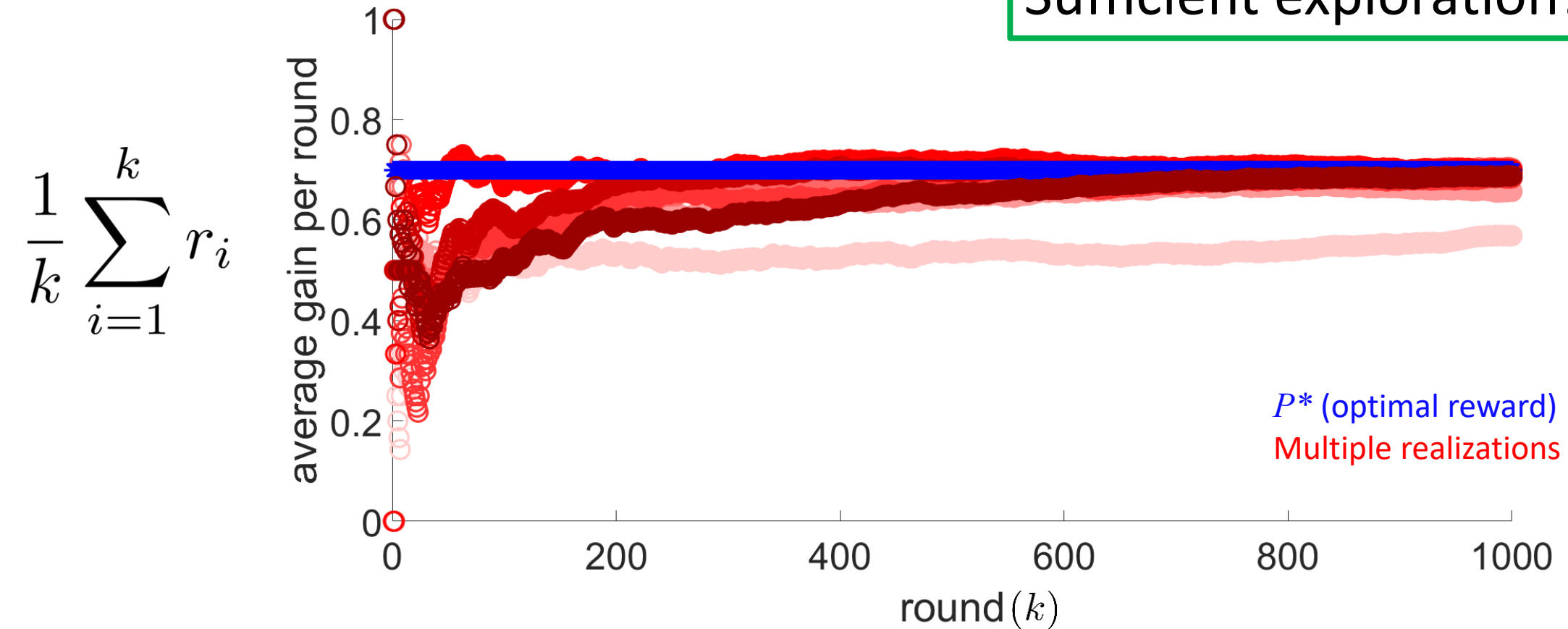
# Example: $\varepsilon$ - Greedy Choice ( $\varepsilon = 0.01$ )

Too little exploration!



# Example: $\varepsilon$ - Greedy Choice ( $\varepsilon = 0.1$ )

Sufficient exploration!



# Reinforcement Learning

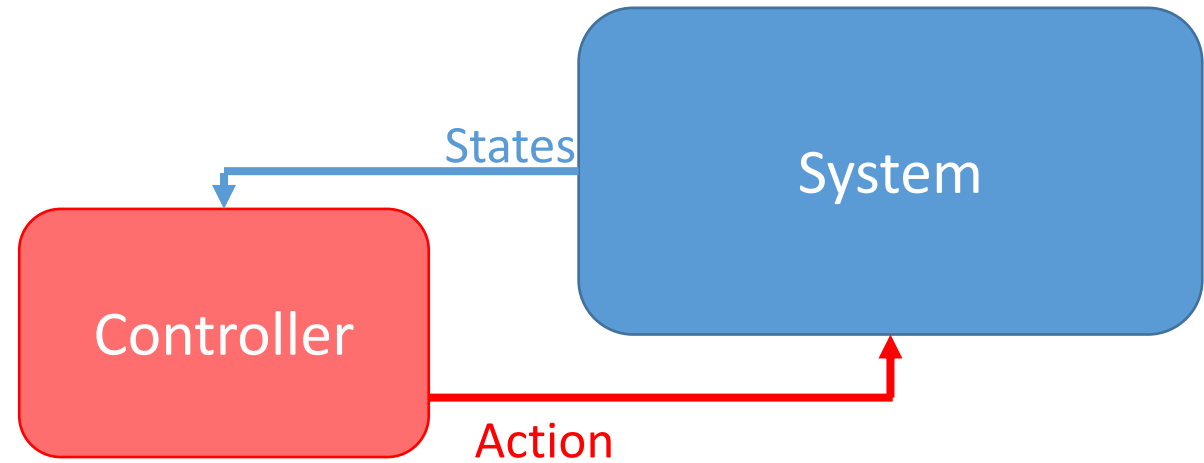
What is Reinforcement Learning?

Multi-Armed Bandits

The Reinforcement Learning Problem

# Key Ingredients

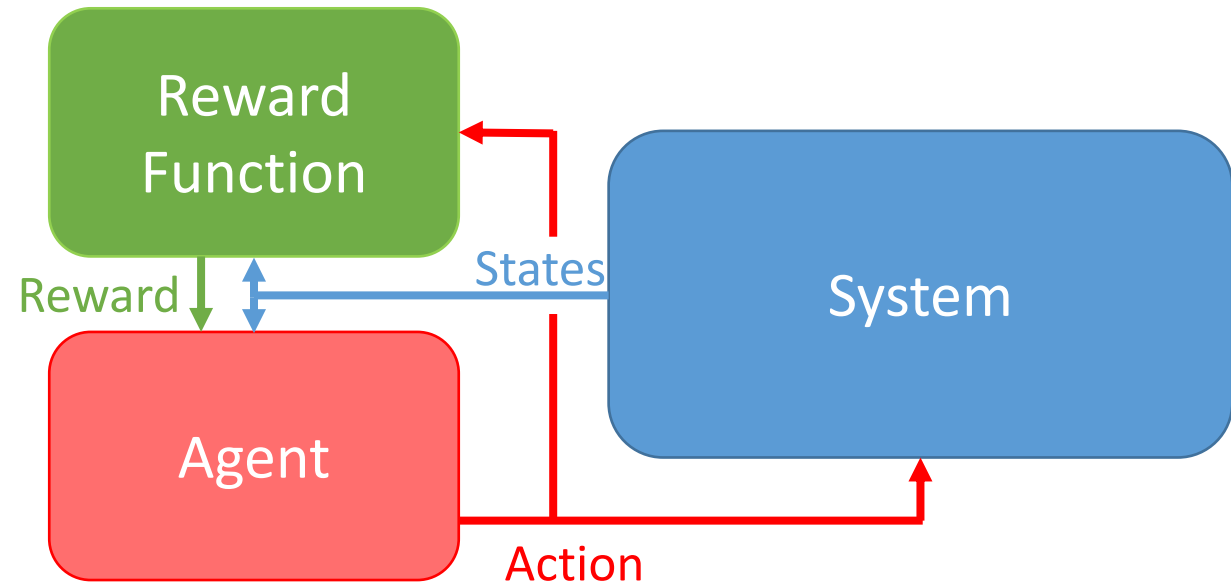
Controller interacts with the system through **States** and **Actions**



# Key Ingredients

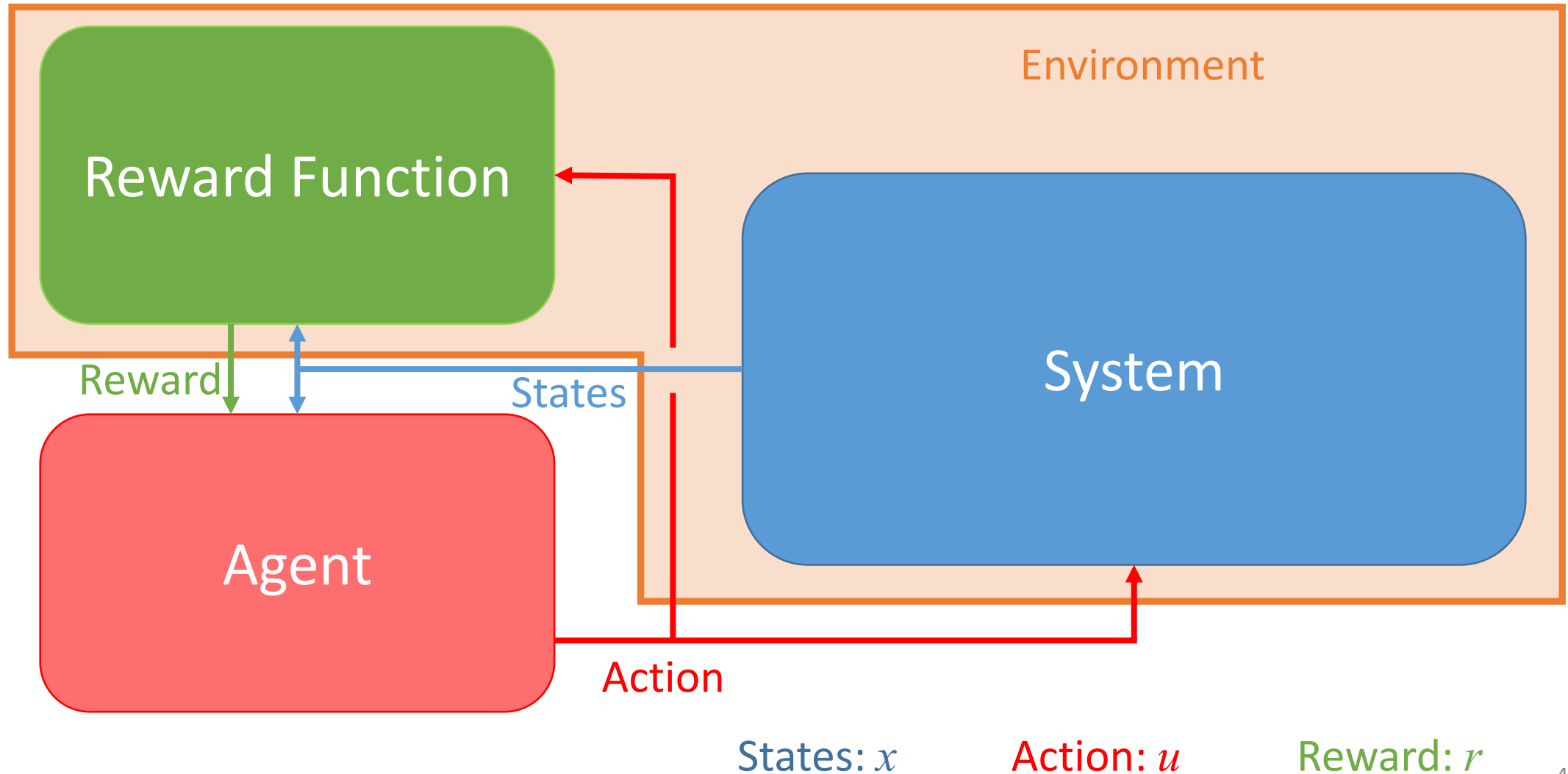
Agent interacts with the system through **States** and **Actions**

Receive **Reward** as a performance feedback

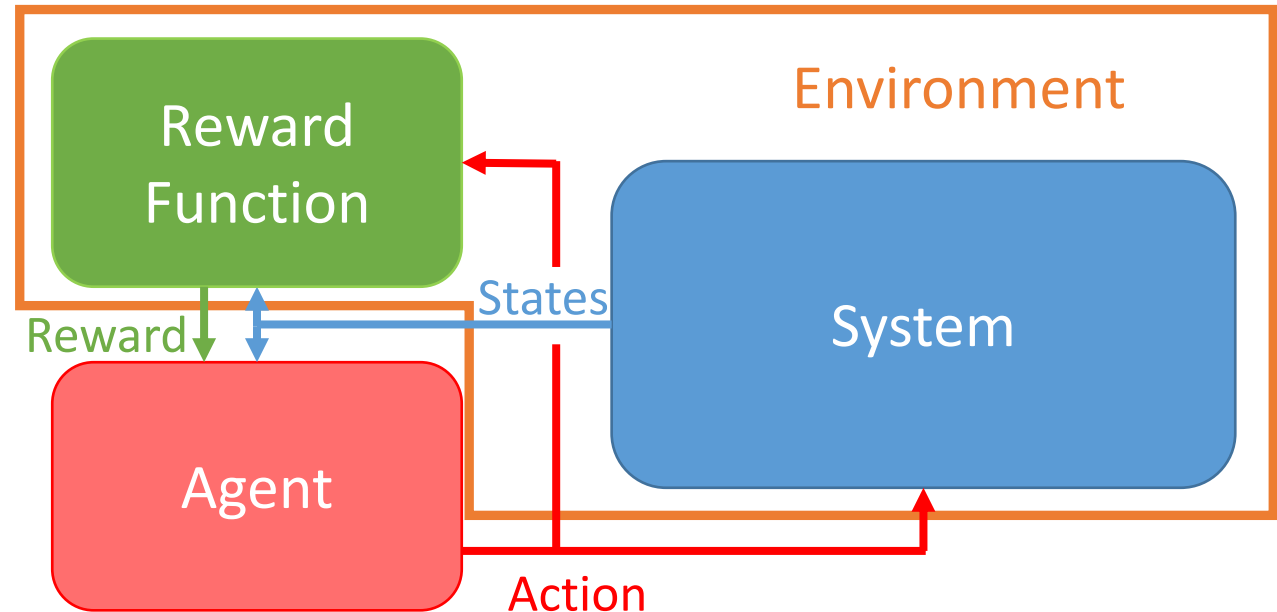




# Key Ingredients



# Agent



The **Agent** is a state feedback controller:

Learns optimal mapping from **States** to **Action** probability

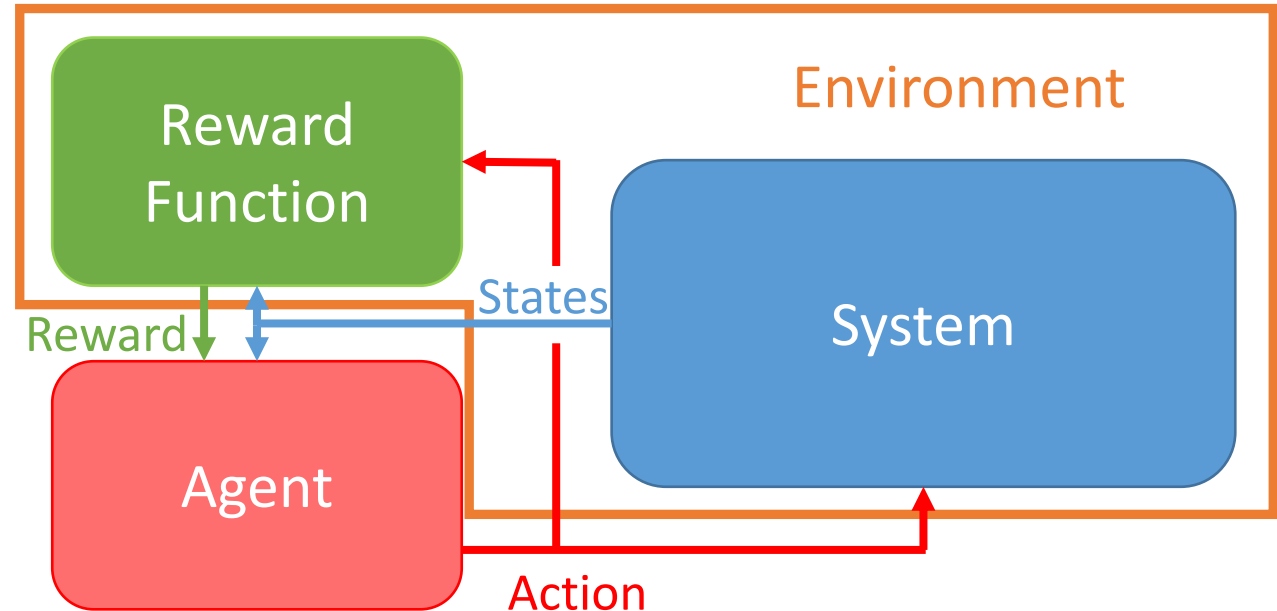
**Policy**  $\pi : X \rightarrow U$  is the control law (deterministic)

(probabilistic)  $\pi(u|x)$  is the probability that the action  $u$  occurs if the state  $= x$

$X$  finite state space

$U$  finite action space

# Goals and Rewards



**Goal** can be formalized as the maximization of the expected value of the cumulative sum of a received scalar signal (**Reward** signal)

**Maximize the Expected Reward**

# Goals and Rewards

Finite Time Horizon: Expected Cumulative Return

$$G_k = \mathbb{E}_\pi \left\{ \sum_{j=k+1}^N r_j \right\} \rightarrow \text{Becomes infinity for } t \text{ growing to infinity}$$

Infinite Time Horizon: Expected Discounted Return

$$G_k = \mathbb{E}_\pi \left\{ \sum_{j=0}^{\infty} \gamma^j r_{k+j+1} \right\} \quad 0 \leq \gamma < 1$$

# Goals and Rewards

Infinite Time Horizon: Expected Discounted Return

$$G_k = \mathbb{E}_\pi \left\{ \sum_{j=0}^{\infty} \gamma^j r_{k+j+1} \right\} \quad 0 \leq \gamma < 1$$

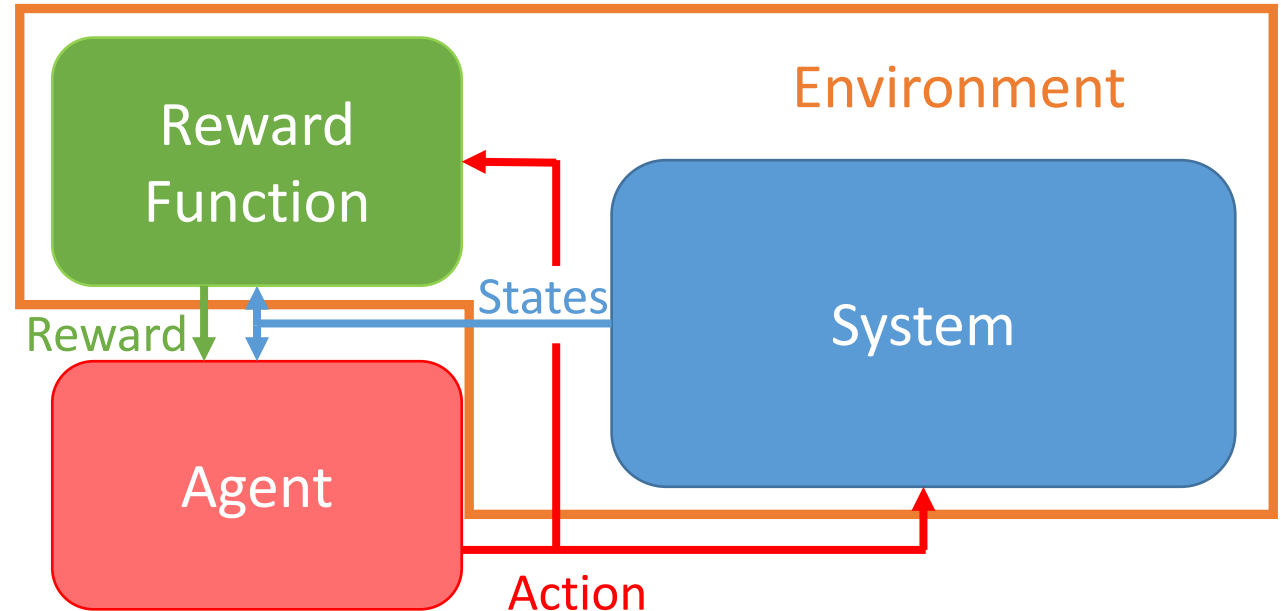
$\gamma = 0$  only care about immediate reward,

$\gamma \approx 1$  future rewards are strongly accounted for

Bounds infinite sum

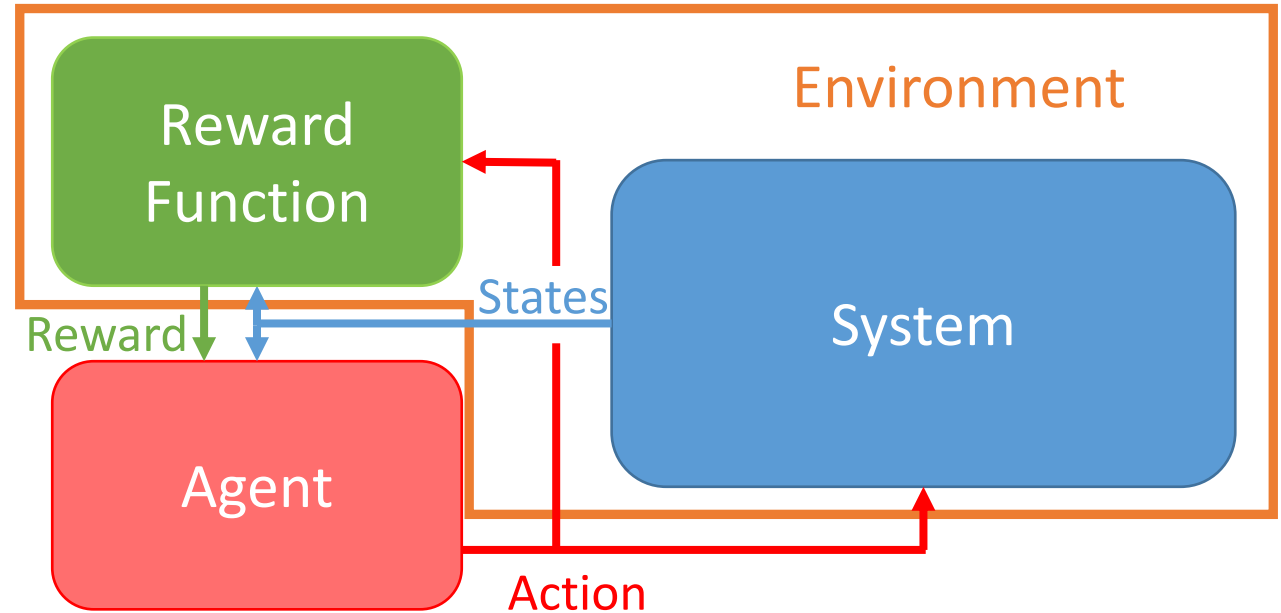
Encodes increasing uncertainty on the future

# Environment



The **Agent** interacts with the **Environment** at a sequence of discrete time steps. Based on its **Actions** the **Agent** receives some representation of the **System State** and a numerical **Reward**.

# Environment



The **Environment** is represented by a Markov Decision Process (MDP - stochastic), or deterministic finite automaton (DFA) in the deterministic case

# Environment (deterministic)

A finite **deterministic finite automaton (DFA)** is a tuple  $\langle X, U, g, \rho \rangle$  where:

$X$  is the finite state space

$U$  is the finite action space

$g : X \times U \rightarrow X$  is the state transition function

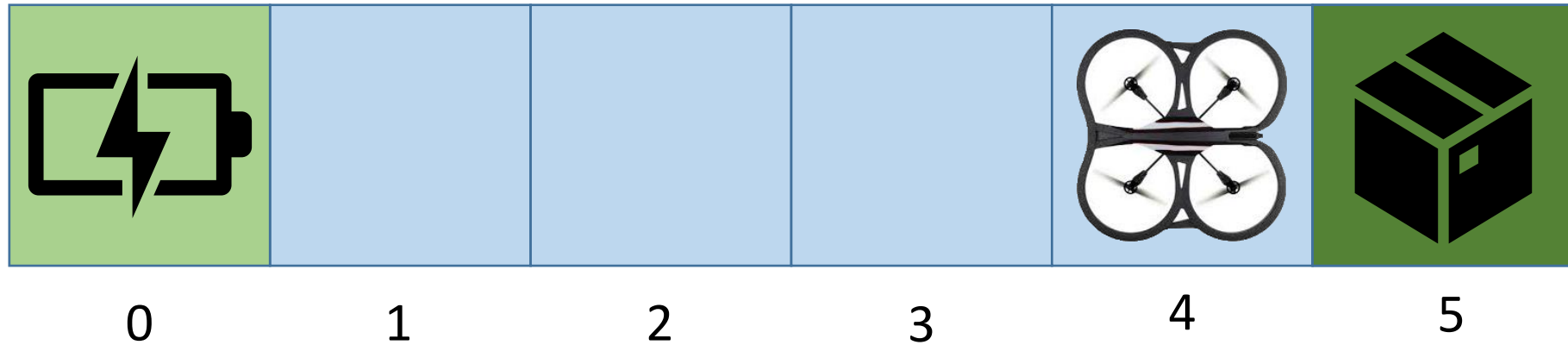
$\rho : X \times U \rightarrow \mathbb{R}$  is the reward function

$x_{k+1} = g(x_k, u_k)$ ,  $r_{k+1} = \rho(x_k, u_k)$  with  $k$  being the discrete time

A **finite DFA** has finite state, action and reward sets.



# Example (deterministic)

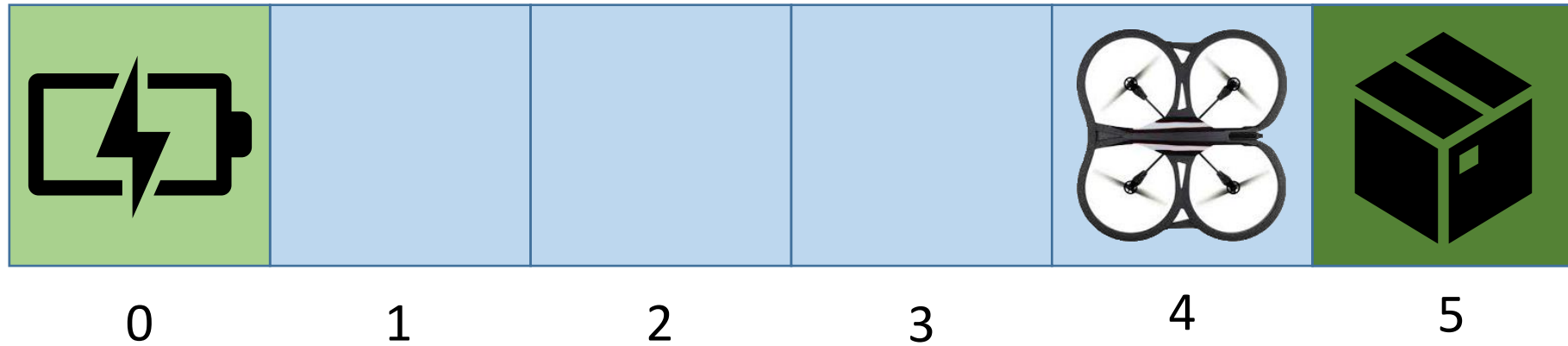


## 1D Package Delivery Drone

Goal: pick up package (+5) or power up (+1)

Episode terminates after reaching one of the goals

# Example (deterministic)



State: Drone Position

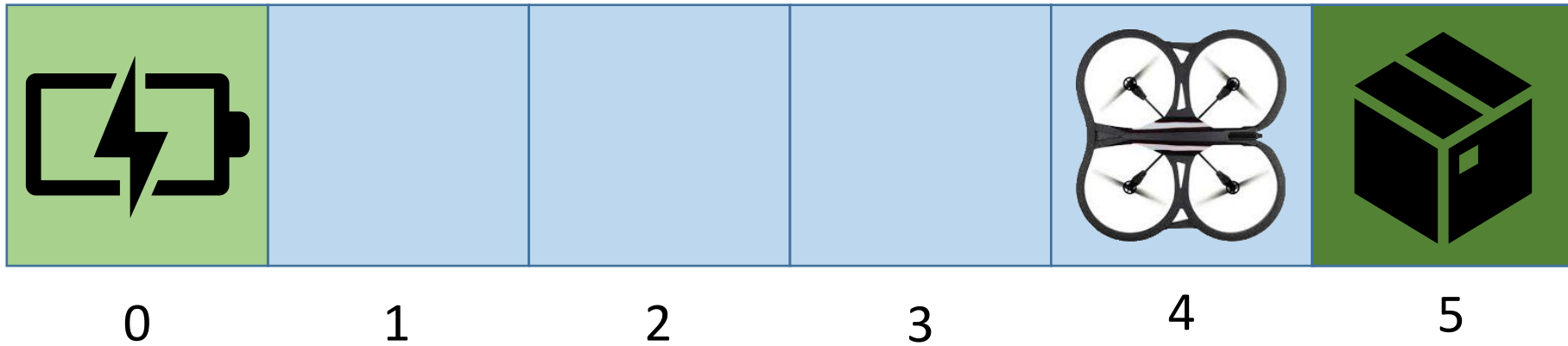
Actions: Move Left, Move Right

Rewards: Collect Package (+5), Power Up (+1)

$$X = \{0, 1, 2, 3, 4, 5\}$$

$$U = \{-1, 1\}$$

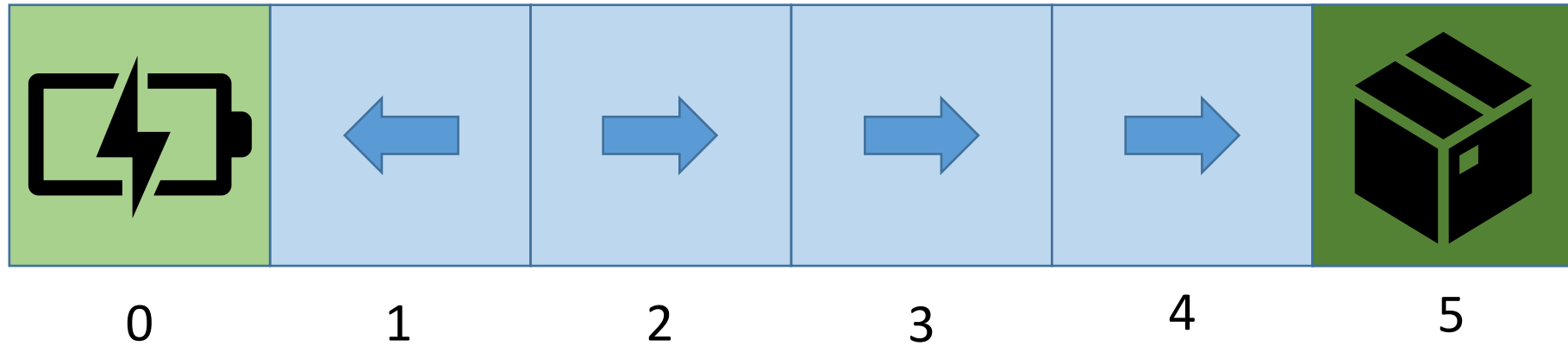
# Example (deterministic)



**State Transition Function**  $x_{k+1} = g(x_k, u_k) = \begin{cases} x_k & \text{if } x \text{ is terminal (0 or 5)} \\ x_k + u_k & \text{otherwise} \end{cases}$

**Reward Function**  $r_{k+1} = \rho(x_k, u_k) = \begin{cases} 1 & \text{if } x = 1 \text{ \& } u = -1 \text{ (power up)} \\ 5 & \text{if } x = 4 \text{ \& } u = 1 \text{ (package)} \\ 0 & \end{cases}$

# Example (deterministic)

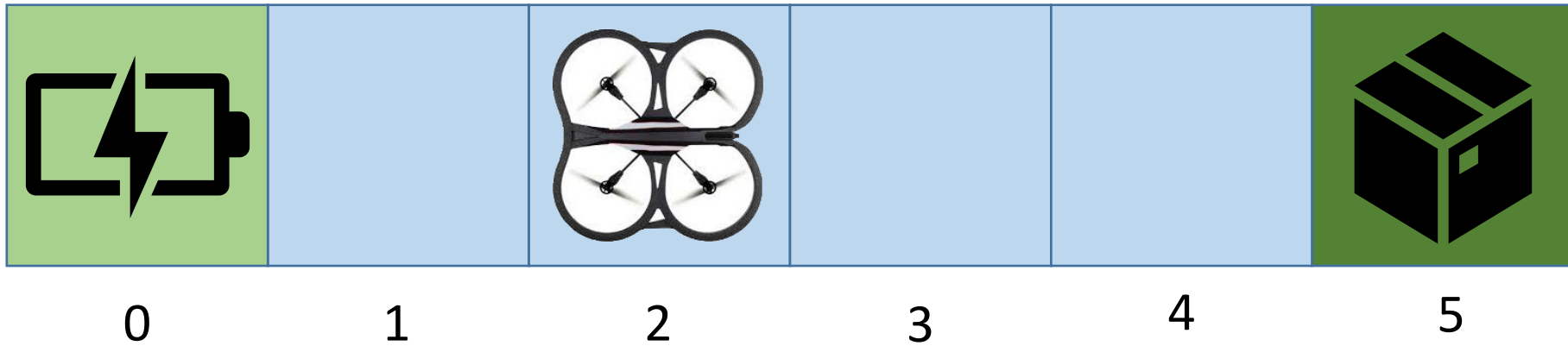


Policy

|               |              |              |
|---------------|--------------|--------------|
| $\pi(0) = *$  | $\pi(2) = 1$ | $\pi(4) = 1$ |
| $\pi(1) = -1$ | $\pi(3) = 1$ | $\pi(5) = *$ |

# Example (deterministic)

$$r_1 = 0 \quad r_2 = 0 \quad r_3 = 5$$



Policy

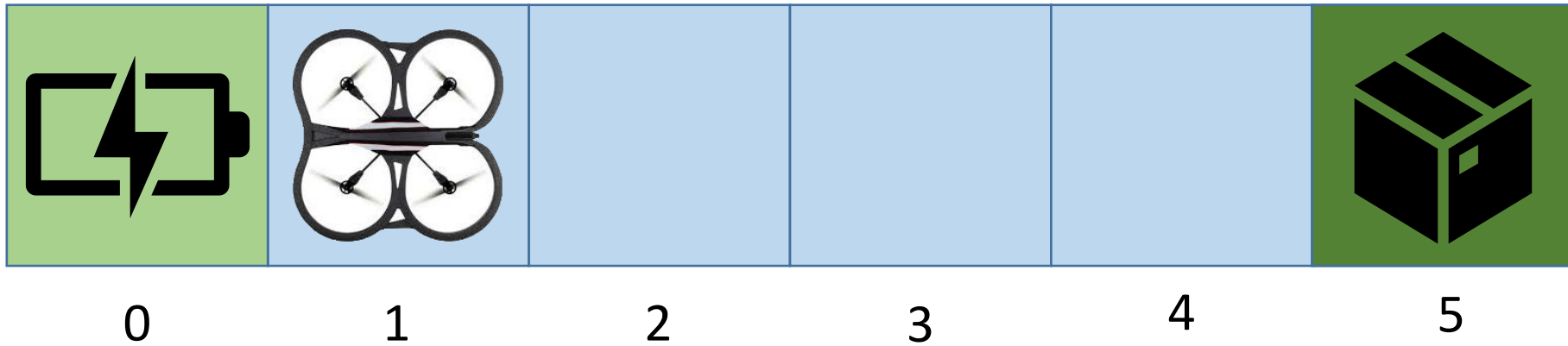
$$\begin{array}{lll} \pi(0) = * & \pi(2) = 1 & \pi(4) = 1 \\ \pi(1) = -1 & \pi(3) = 1 & \pi(5) = * \end{array}$$

Discounted reward

$$\gamma = 0.5 \quad G_k = \mathbb{E}_\pi \left\{ \sum_{j=0}^{\infty} \gamma^j r_{k+j+1} \right\} = 1 \times 0 + 0.5 \times 0 + 0.5^2 \times 5$$

# Example (deterministic)

$$r_1 = 1$$



Policy

$$\begin{array}{lll} \pi(0) = * & \pi(2) = 1 & \pi(4) = 1 \\ \pi(1) = -1 & \pi(3) = 1 & \pi(5) = * \end{array}$$

Discounted reward

$$\gamma = 0.5 \quad G_k = \mathbb{E}_\pi \left\{ \sum_{j=0}^{\infty} \gamma^j r_{k+j+1} \right\} = 1 \times 1$$

# Reinforcement Learning – After the Break

Stochastic Environment

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning

# Reinforcement Learning

Stochastic Environment

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning



# Environment (deterministic)

A finite **deterministic finite automaton (DFA)** is a tuple  $\langle X, U, g, \rho \rangle$  where:

$X$  is the finite state space

$U$  is the finite action space

$g : X \times U \rightarrow X$  is the state transition function

$\rho : X \times U \rightarrow \mathbb{R}$  is the reward function

$x_{k+1} = g(x_k, u_k)$ ,  $r_{k+1} = \rho(x_k, u_k)$  with  $k$  being the discrete time

A **DFA** has finite state, action and reward sets.

# Environment (stochastic)

A finite **Markov Decision Process (MDP)** is a tuple:  $\langle X, U, p_{\text{tran}}, \rho \rangle$

$p(x_{k+1}, r_{k+1} | x_k, u_k)$  the **joint state transition and reward probability**

**Transition probability:**  $p_{\text{tran}}(x_{k+1} | x_k, u_k) = \sum_{r \in R} p(x_{k+1}, r | x_k, u_k)$

**Expected reward:**  $\rho(x_k, u_k) = \sum_{r \in R} r \sum_{x \in X} p(x, r | x_k, u_k)$

$X, U, R$  are the finite state, action and reward space respectively  
and  $k$  is the discrete time

# Environment (stochastic)

A finite **Markov Decision Process (MDP)** is a tuple:  $\langle X, U, p_{\text{tran}}, \rho \rangle$

$p(x_{k+1}, r_{k+1} | x_k, u_k)$  the **joint state transition and reward probability**

**Transition probability:**  $p_{\text{tran}}(x_{k+1} | x_k, u_k) = \sum_{r \in R} p(x_{k+1}, r | x_k, u_k)$

**Expected reward:**  $\rho(x_k, u_k, x_{k+1}) = \sum_{r \in R} r \frac{p(x_{k+1}, r | x_k, u_k)}{p_{\text{tran}}(x_{k+1} | x_k, u_k)}$

$X, U, R$  are the finite state, action and reward space respectively  
and  $k$  is the discrete time

The expected reward can also depend on the next state

# Recycling Robot Example (stochastic)

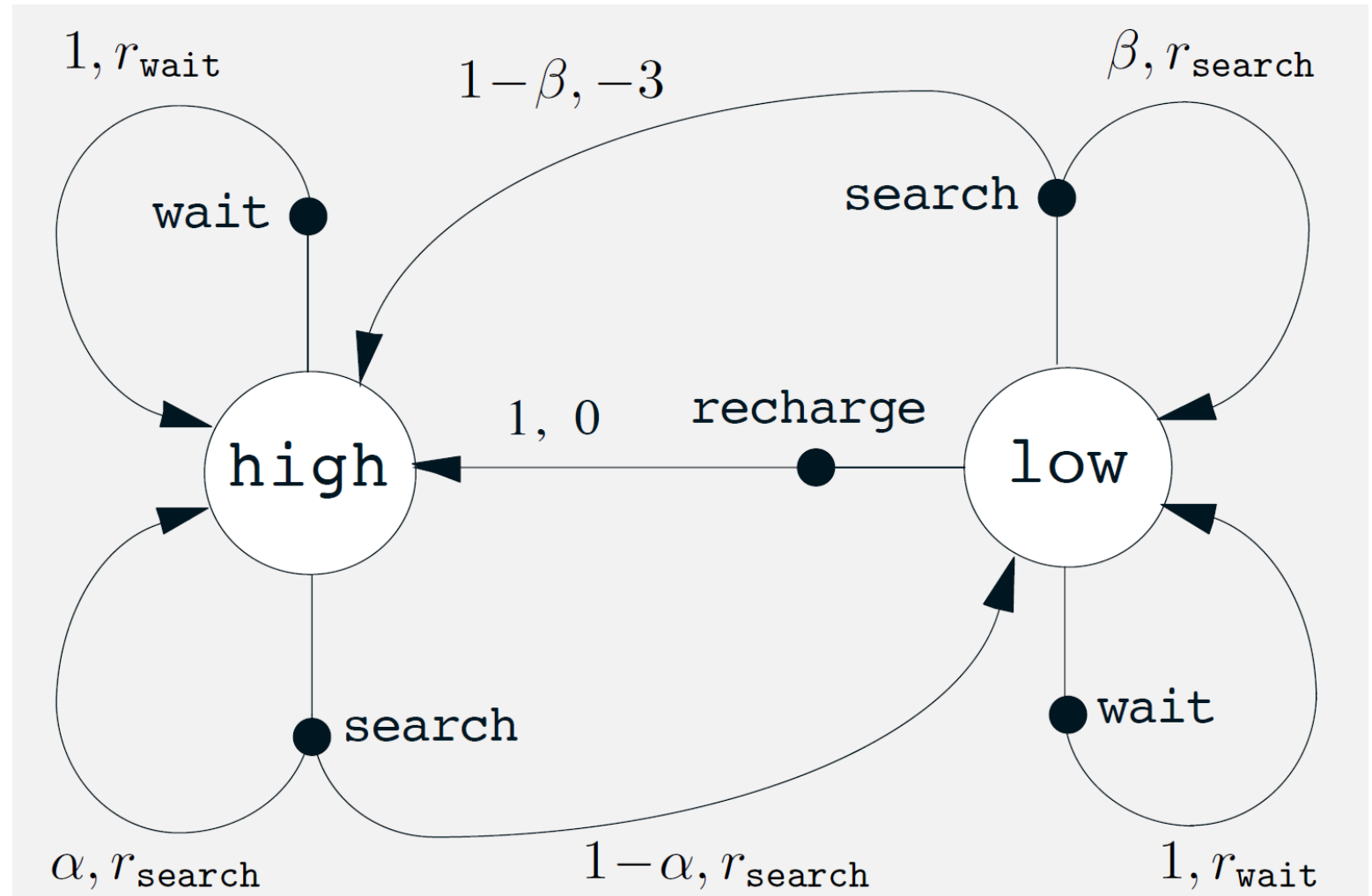
Mobile robot collects empty soda cans in an environment.

Battery: High or Low

Activity: Search, Wait, Recharge

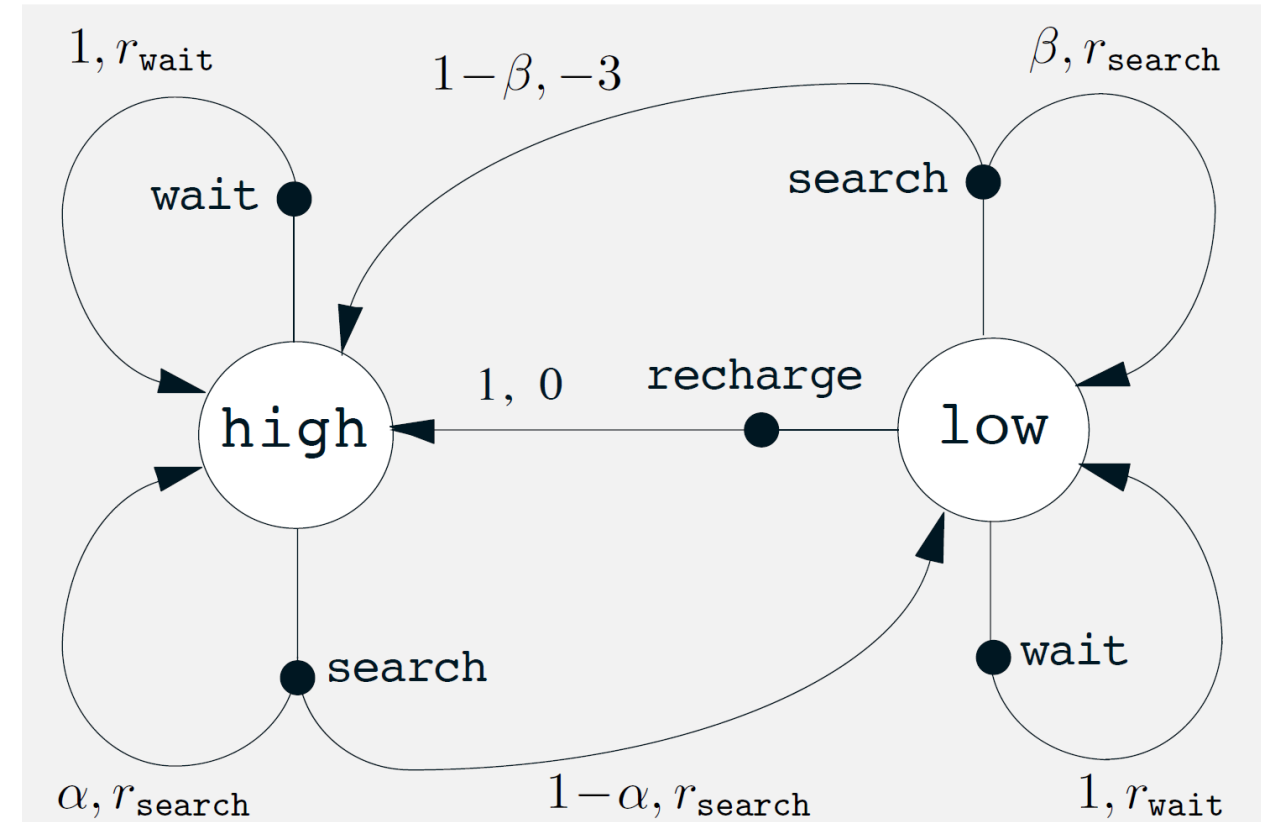
Reward highest when actively searching, but depletes battery

When battery low, there is a risk of running empty, resulting in a large penalty.



# Recycling Robot Example (stochastic)

| $x_k$ | $u_k$    | $x_{k+1}$ | $p(x_{k+1} x_k, u_k)$ | $\rho(x_k, u_k, x_{k+1})$ |
|-------|----------|-----------|-----------------------|---------------------------|
| High  | Search   | High      | $\alpha$              | $r_{\text{search}}$       |
| High  | Search   | Low       | $1-\alpha$            | $r_{\text{search}}$       |
| Low   | Search   | High      | $1-\beta$             | -3                        |
| Low   | Search   | Low       | $\beta$               | $r_{\text{search}}$       |
| High  | Wait     | High      | 1                     | $r_{\text{wait}}$         |
| High  | Wait     | Low       | 0                     | -                         |
| Low   | Wait     | High      | 0                     | -                         |
| Low   | Wait     | Low       | 1                     | $r_{\text{wait}}$         |
| Low   | Recharge | High      | 1                     | 0                         |
| Low   | Recharge | Low       | 0                     | -                         |



Source: R.S. Sutton and A.G. Barto, "Reinforcement Learning: An Introduction", The MIT Press, 2018

# Reinforcement Learning

Stochastic Environment

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning

# Value Function

Discounted Return:  $G_k = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \right\} \quad 0 \leq \gamma < 1$

Value Function:  $V_\pi(x_k) = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k \right\}$

Expected discounted return given the system states at time  $k$  for policy  $\pi$

# Q-Function

Discounted Return:  $G_k = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \right\} \quad 0 \leq \gamma < 1$

Value Function:  $V_\pi(x_k) = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k \right\}$

Q-Function:  $Q_\pi(x_k, u_k) = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k, u_k \right\}$

Expected discounted return given the system states and the action taken at time  $k$  for policy  $\pi$



# Q-Function

Q-Function:

$$Q_{\pi}(x_k, u_k) = \mathbb{E}_{\pi} \left\{ \sum_{\tau=0}^{\infty} \gamma^{\tau} r_{k+\tau+1} \middle| x_k, u_k \right\}$$

Expected discounted return given the system states and the action taken at time  $k$  for policy  $\pi$

Action at time  $k$  does not need to follow the policy  $\pi$ , all the consecutive actions are taken following the policy  $\pi$

# Bellman Equation

Develop Q-Function one step ahead:

$$\begin{aligned} Q_{\pi}(x_k, u_k) &= \mathbb{E}_{\pi} \left\{ \sum_{\tau=0}^{\infty} \gamma^{\tau} r_{k+\tau+1} \middle| x_k, u_k \right\} \\ &= \mathbb{E}_{\pi} \left\{ r_{k+1} + \sum_{\tau=1}^{\infty} \gamma^{\tau} r_{k+\tau+1} \middle| x_k, u_k \right\} \\ &= \mathbb{E}_{\pi} \{ r_{k+1} + \gamma V_{\pi}(x_{k+1}) \mid x_k, u_k \} \end{aligned}$$

Link the present with the future

# Optimal Policy

Optimality of a policy is given by the Value Function:

$$\pi \geq \pi' \iff V_{\pi}(x) \geq V_{\pi'}(x) \forall x \in X$$

Policy  $\pi$  is better than policy  $\pi'$  if and only if its expected return is greater than or equal to that of  $\pi'$  for all states.

# Optimal Policy

Optimality of a policy is given by the Value Function:

$$\pi \geq \pi' \iff V_\pi(x) \geq V_{\pi'}(x) \forall x \in X$$

Policy  $\pi$  is better than policy  $\pi'$  if and only if its expected return is greater than or equal to that of  $\pi'$  for all states.

Multiple optimal policies  $\pi_*(x)$  can exist, they share the same optimal Value Function and optimal Q-Function:

$$V_*(x) = \max_{\pi} V_\pi(x)$$
$$Q_*(x, u) = \max_{\pi} Q_\pi(x, u)$$

The optimal policy  $\pi$  can be retrieved from the optimal Q-function by selecting the action that maximized the Q-function at a given state.

# Bellman Equation of Optimality

Value Function:  $V_*(x_k) = \max_u Q_*(x_k, u)$

$$= \max_u \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k, u_k \right\}$$

$$= \max_u \mathbb{E}_\pi \{ r_{k+1} + \gamma V_*(x_{k+1}) | x_k, u_k \}$$

$$= \max_u \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u) (r + \gamma V_*(x_{k+1}))$$

# Bellman Equation of Optimality

Q-Function:

$$\begin{aligned} Q_*(x_k, u_k) &= \mathbb{E}_\pi \left\{ r_{k+1} + \sum_{\tau=1}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k, u_k \right\} \\ &= \mathbb{E}_\pi \{ r_{k+1} + \gamma V_*(x_{k+1}) | x_k, u_k \} \\ &= \mathbb{E}_\pi \left\{ r_{k+1} + \gamma \max_u Q_*(x_{k+1}, u) \middle| x_k, u_k \right\} \\ &= \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q_*(x_{k+1}, u_{k+1}) \right) \end{aligned}$$

# Reinforcement Learning

Stochastic Environment

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning

# Reinforcement Learning

Monte Carlo

Dynamic Programming

Temporal Difference



# Reinforcement Learning

Monte Carlo

Dynamic Programming

Temporal Difference

# Temporal Difference Learning

$$\text{New Estimate} \leftarrow \text{Old Estimate} + \text{Step Size} \times [\text{Target} - \text{Old Estimate}]$$

**Learn directly from raw experience**

**No model** of the system dynamics required

**Update** the knowledge of the agent on **every timestep** (action) rather than on every episode (reaching the goal or end state)  
(update estimates based upon other estimates - ***bootstrapping***)

**Incremental** nature (good for online applications)

# The Prediction Problem

Estimate the Value Function  $V_\pi(x)$  for a given policy  $\pi$ .

New Estimate  $\leftarrow$  Old Estimate + Step Size  $\times$  [Target - Old Estimate]

$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha [G_k - V_\pi(x_k)]$$



$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha [\underbrace{r_{k+1} + \gamma V_\pi(x_{k+1})}_{\text{Temporal Difference}} - V_\pi(x_k)]$$

Temporal Difference

Instead of waiting to see all obtained rewards required for computing  $G_k$   
plug in the estimate  $V_\pi(x_{k+1})$   
 $\rightarrow$  bootstrapping

# Temporal Difference Learning

**Input:** the policy  $\pi$  to be evaluated

**Parameters:** step size  $\alpha \in (0, 1]$

**Initialize**  $V_\pi(x)$  arbitrarily,  $V(\text{terminal}) = 0$ .

**for** each episode **do**

    Initialize  $x_0$

**Repeat** for each time step of the episode

        Obtain  $u_k$  based on  $x_k$  using policy  $\pi$

        Take action  $u_k$ , observe  $r_{k+1}, x_{k+1}$

$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha [r_{k+1} + \gamma V_\pi(x_{k+1}) - V_\pi(x_k)]$$

$k = k + 1$

**Until** the states are terminal

**end for**

# Reinforcement Learning

Stochastic Environment

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning

# Temporal Difference Learning for Control

**SARSA** (State-Action-Reward-State-Action)

on-policy TD control

**Q-Learning**

off-policy TD Control

An **off-policy** learner learns the value of the optimal policy independently of the agent's actions.

An **on-policy** learner learns the value of the policy being carried out by the agent including the exploration steps.

# Temporal Difference Learning for Control

**SARSA** (State-Action-Reward-State-Action)

on-policy TD control

**Q-Learning**

off-policy TD Control

An **off-policy** learner learns the value of the optimal policy independently of the agent's actions.

An **on-policy** learner learns the value of the policy being carried out by the agent including the exploration steps.

# Q-Learning

Directly learn the optimal Q-Function  $Q_*(x_t, u_t)$ , use it to compute  $\pi_*$

Independent of the policy being followed

Policy still is important:

- Determines which state-action pairs are visited and updated

- Exploration required



# Q-Learning

Take **Bellman optimality equation** at some state and action

$$Q_*(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q_*(x_{k+1}, u_{k+1}) \right)$$

# Q-Learning

Take **Bellman optimality equation** at some state and action

$$Q_*(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q_*(x_{k+1}, u_{k+1}) \right)$$

Turn into **iterative update**

$$Q(x_k, u_k) \leftarrow \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1}) \right)$$

# Q-Learning

Take **Bellman optimality equation** at some state and action

$$Q_*(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q_*(x_{k+1}, u_{k+1}) \right)$$

Turn into **iterative update**

$$Q(x_k, u_k) \leftarrow \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left( r + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1}) \right)$$

Instead of a transition model, use the **transition sample** at each step

$$Q(x_k, u_k) \leftarrow r_{k+1} + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1})$$

# Q-Learning

Instead of a transition model, use the **transition sample** at each step

$$Q(x_k, u_k) \leftarrow r_{k+1} + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1})$$

Make update incremental

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha \left( \underbrace{r_{k+1} + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k)}_{\text{Temporal Difference}} \right)$$

Learning rate  $\alpha \in (0, 1]$

Temporal Difference

# Q-Learning

**Parameters:** step size  $\alpha \in (0, 1]$  and  $0 < \epsilon < 1$

**Initialize**

$Q(x, u)$  arbitrarily for all possible states and actions,

$Q(\text{terminal}, \cdot) = 0$ .

**for** each episode **do**

Initialize  $x_0$

**Repeat** for each time step of the episode

Obtain  $u_k$  based on  $x_k$  using policy  $\pi$  derived from  $Q$   
(e.g.  $\epsilon$ -greedy)

Take action  $u_k$ , observe  $r_{k+1}, x_{k+1}$

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha [r_{k+1} + \gamma \max_u Q(x_{k+1}, u) - Q(x_k, u_k)]$$

$$k = k + 1$$

**Until** the states are terminal

**end for**

# Q-Learning

**Parameters:** step size  $\alpha \in (0, 1]$  and  $0 < \epsilon < 1$

**Initialize**

$Q(x, u)$  arbitrarily for all possible states and actions,

$Q(\text{terminal}, \cdot) = 0$ .

**for** each episode **do**

Initialize  $x_0$

**Repeat** for each time step of the episode

Obtain  $u_k$  based on  $x_k$  using policy  $\pi$  derived from  $Q$   
(e.g.  $\epsilon$ -greedy)

Take action  $u_k$ , observe  $r_{k+1}, x_{k+1}$

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha [r_{k+1} + \gamma \max_u Q(x_{k+1}, u) - Q(x_k, u_k)]$$

$k = k + 1$

**Until** the states are terminal

**end for**

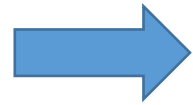
In off-policy learning any policy can be used (exploration recommended), the choice displayed here is one of the more common choices

# Exploration - Exploitation

Essential for **convergence** to  $Q_*(x_k, u_k)$

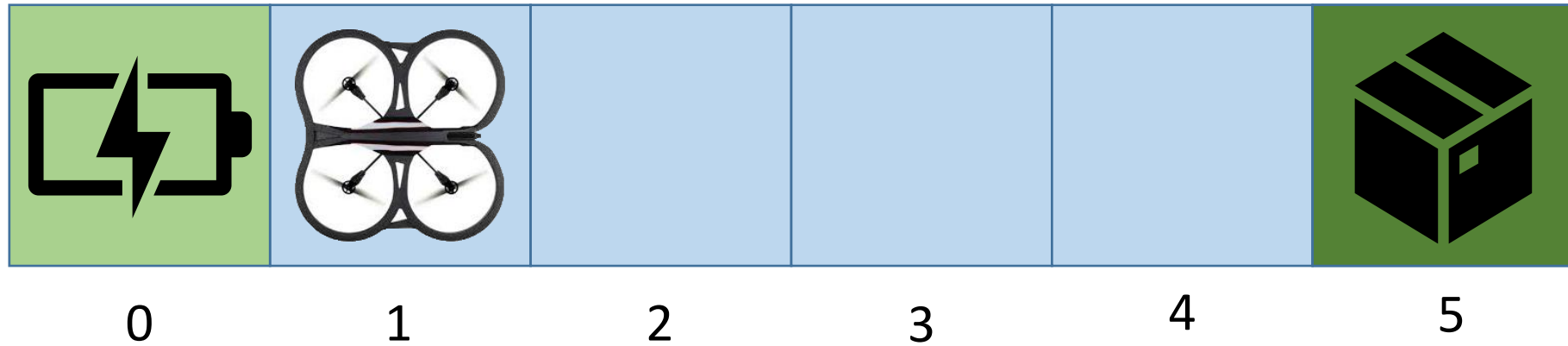
all  $(x, u)$  pairs need to be visited infinitely often

**Exploration** necessary, but **exploitation** also necessary:



$\epsilon$  -greedy approaches are recommended

# Delivery Drone Example



Parameters:

$$\alpha = 0.2$$

$$\varepsilon = 0.3$$

$$\gamma = 0.5$$

$$x_0 = 2 \text{ or } 3$$



Iteration: 1, step: 0

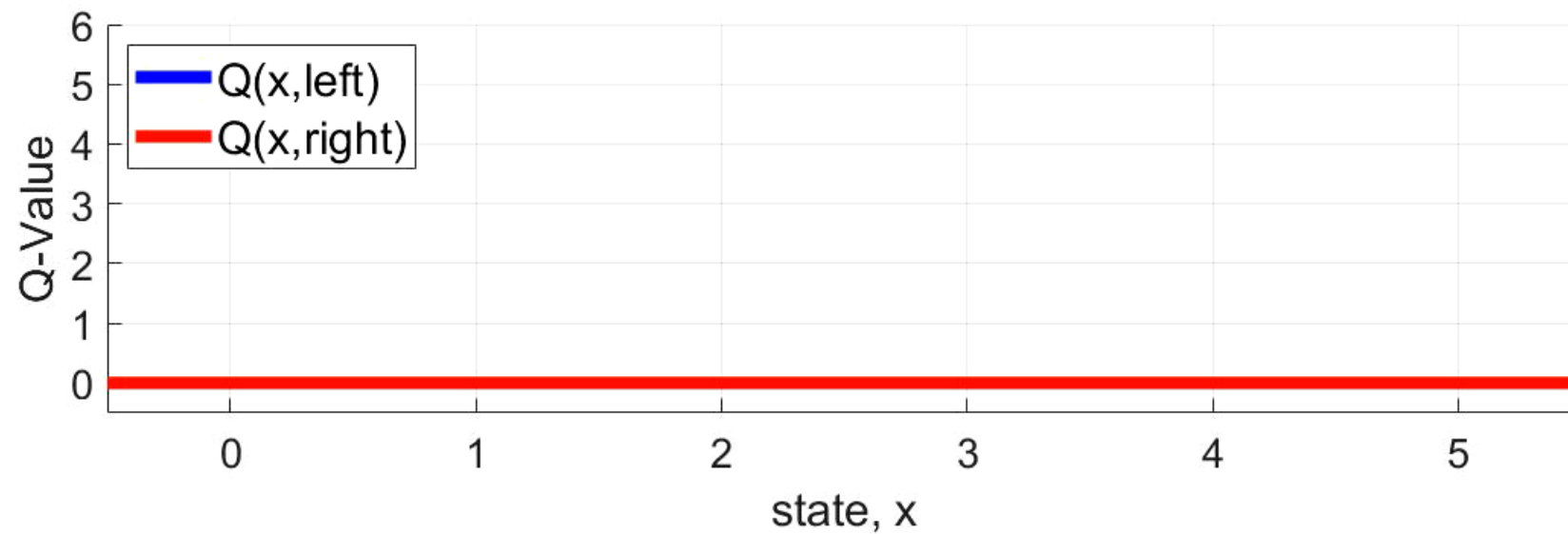
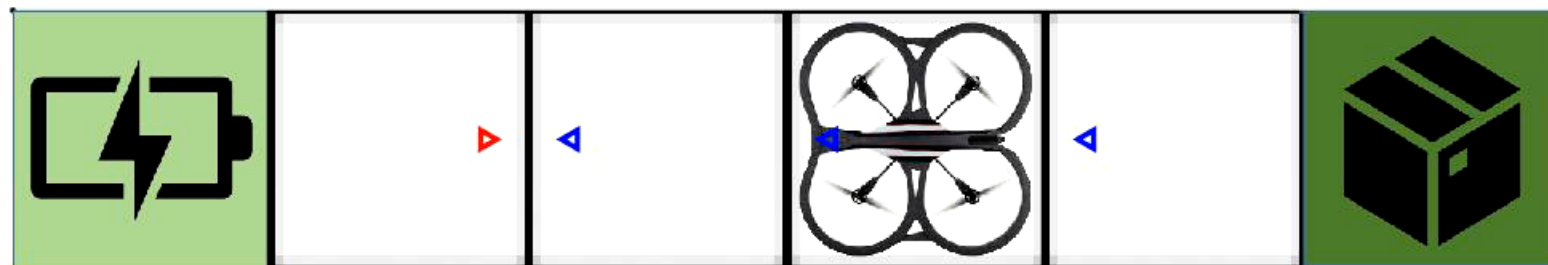
Parameters:

$$\alpha = 0.2$$

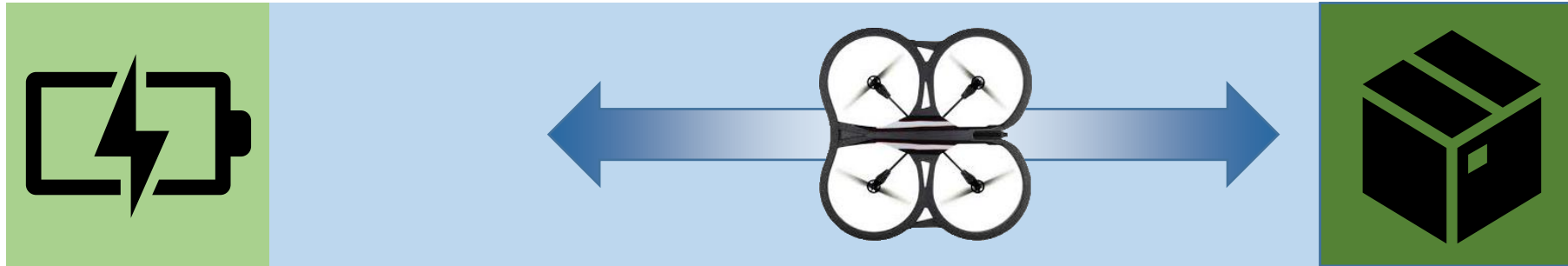
$$\varepsilon = 0.3$$

$$\gamma = 0.5$$

$$x_0 = 2 \text{ or } 3$$



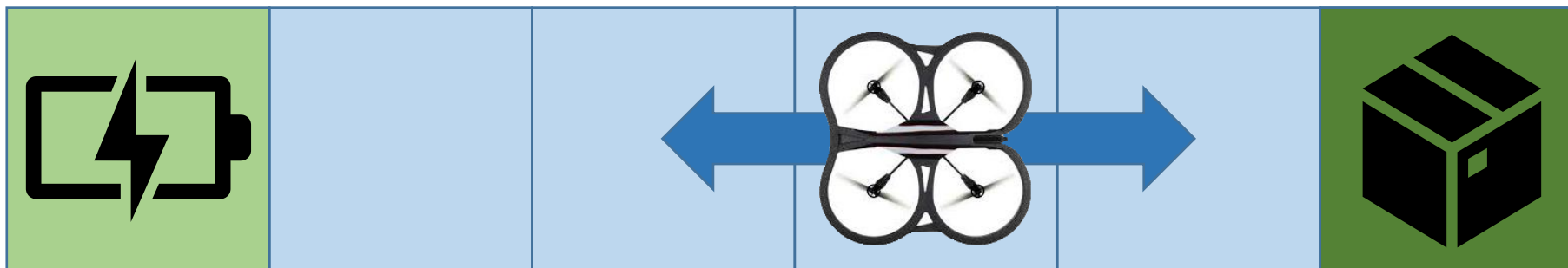
# State / Action Discretization



Continuous position

Continuous velocity

Discretization



Gridded position

Fixed velocity

0

1

2

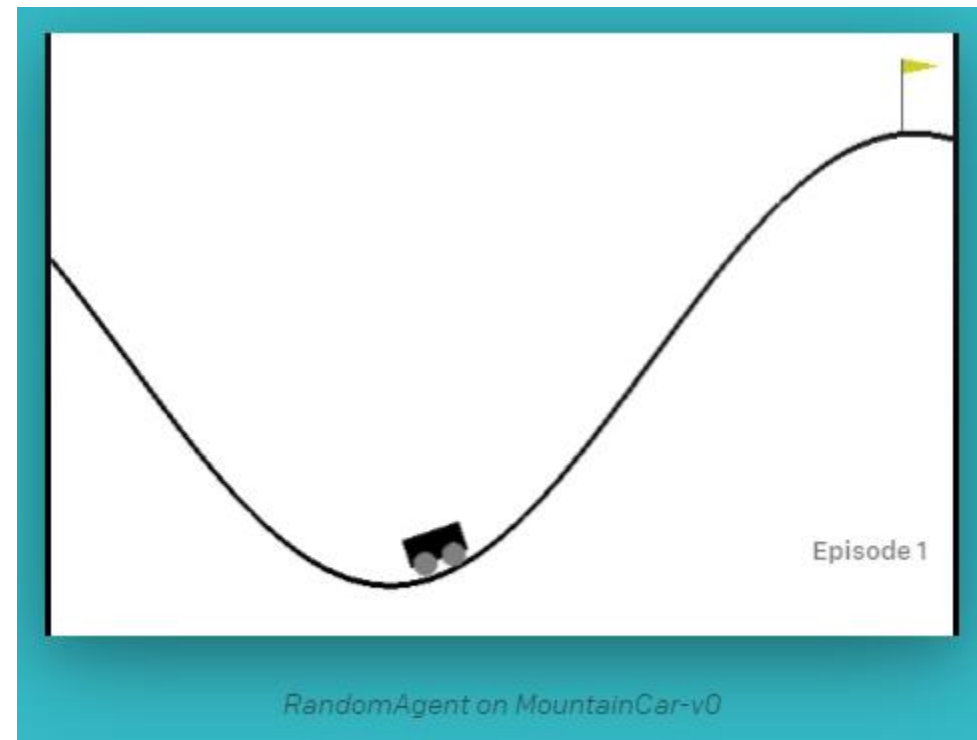
3

4

5

# State / Action Discretization

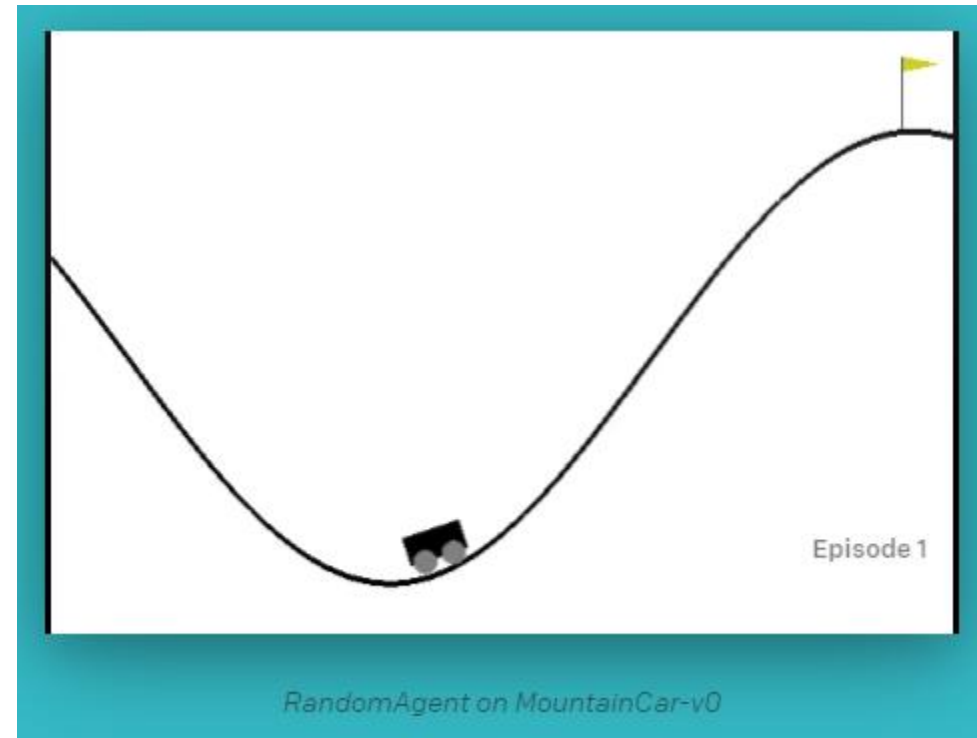
A car is on a one-dimensional track, positioned between two "mountains". The goal is to drive up the mountain on the right; however, the car's engine is not strong enough to scale the mountain in a single pass. Therefore, the only way to succeed is to drive back and forth to build up momentum.



# State / Action Discretization

Action Space?

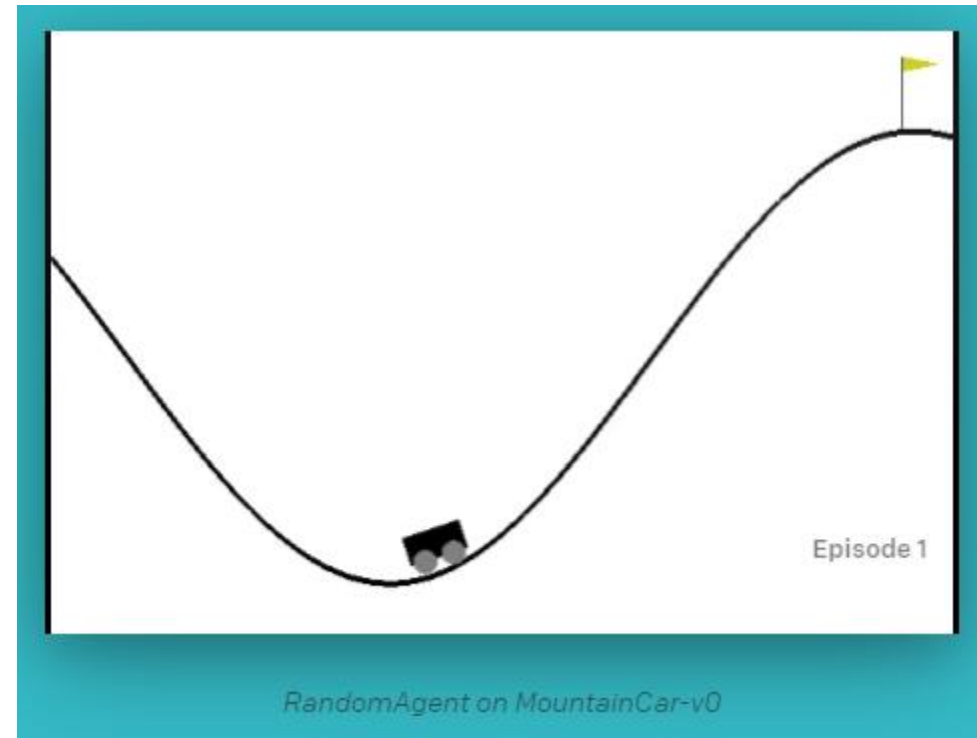
State Space?



# State / Action Discretization

Action Space Discretization?

State Space Discretization?

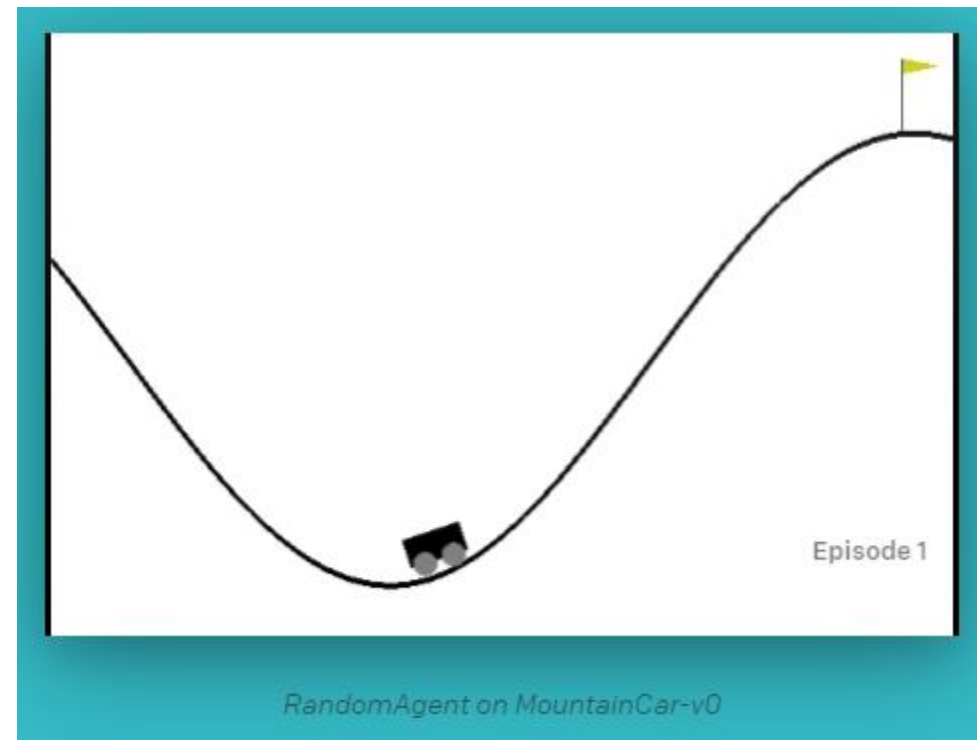


# State / Action Discretization

Action Space Discretization?

State Space Discretization?

Best to discretize values in a normalized way  
(for training purposes later)



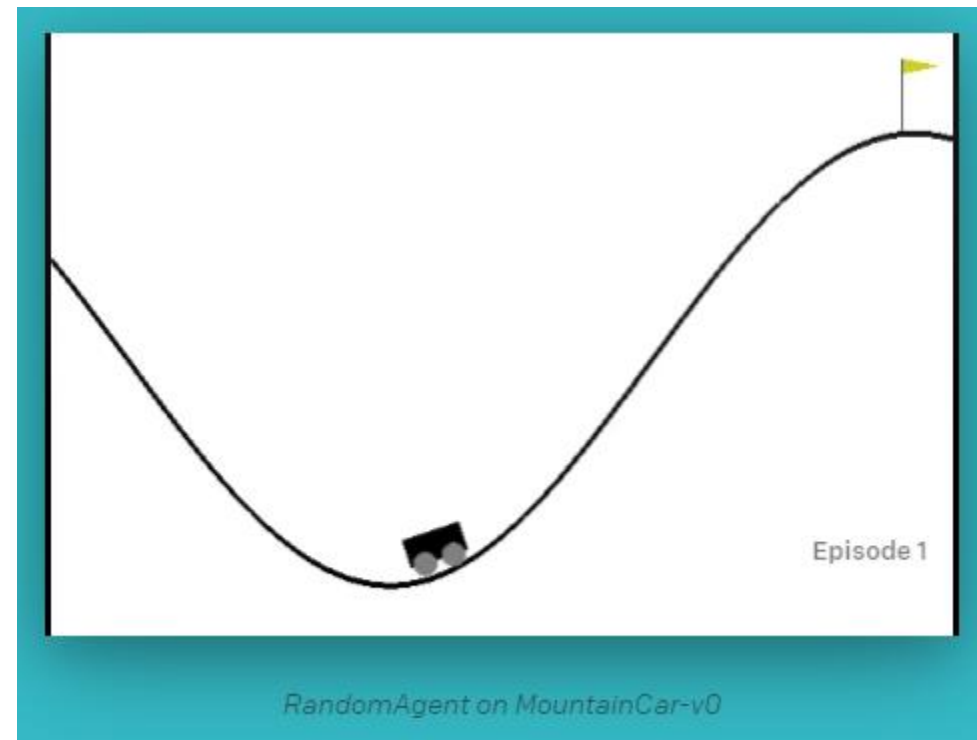
Too coarse discretization → performance drop  
Too fine discretization → large space to explore



$n_x$  continuous states,  $n$  discretization grid  
→  $n^{n_x}$  discrete states

# State / Action Discretization

Solve in the exercises!



# Reinforcement Learning

What is Reinforcement Learning?

Multi-Armed Bandits

Finite Markov Decision Process (Deterministic & Stochastic)

Towards Reinforcement Learning:

Value Function, Q-Function, Bellman Equation

Temporal Difference Learning

Q-Learning



# Remaining Problems

Systems with large state space → large tables!

Continuous state space?